

**Centre régional des métiers de l'éducation et
de la formation Marrakech Safi**
**Cycle de qualification des cadres du corps des
enseignants**
Département informatique

**Livret de la didactique
de l'informatique**

Pr. SERRAR Ouafae

Table de matière

Table de matière	2
Histoire et épistémologie de l'informatique	4
I. Histoire de l'informatique :	4
II. L'épistémologie de l'informatique	4
1. L'épistémologie : un outil de compréhension de la discipline	4
2. Les principales approches épistémologiques de l'informatique	5
3. Pour l'enseignant : des implications pratiques directes	8
4. Au-delà de la technique : former des enseignants réflexifs	9
5. Un enjeu de légitimité de la discipline	10
III. Filiation conceptuel de l'informatique	11
1. la relation croisée entre l'informatique et les autres sciences :	11
2. Question de l'identité de l'informatique	12
3. Dimensions de l'informatique.....	12
IV. Concepts de base de l'informatique:	13
V. La dualité d'utilisation de l'informatique : Objet d'enseignement et outil pédagogique	13
La dualité objet/outil : une tension à transformer en richesse.....	14
VI. Modes de pensée en informatique :	14
Introduction : Au-delà de la technique, des façons de penser.....	14
La pensée logique : raisonner avec rigueur	14
La pensée algorithmique : concevoir des procédures efficaces	15
La pensée computationnelle (Computational Thinking).....	16
Synthèse : former des penseurs, pas des codeurs	16
Les théories d'apprentissage	18
I. Behaviorisme:	18
II. Cognitivisme:	18
III. Constructivisme:	19
IV. Connectivisme:	20
V. La théorie de l'apprentissage multimédia:.....	21

Théorie de la charge cognitive	21
VI. La théorie de l'apprentissage multimédia:	23
VII. Conclusion:	23
Didactique de l'informatique : concepts généraux.....	24
I. Didactique vs pédagogie	24
1. Didactique.....	24
2. La pédagogie :	24
II. Différence entre savoir et connaissance:.....	24
III. Triangle didactique	25
1. Définition.....	25
2. Les trois axes du triangle didactique : praxéologie, épistémologie et psychologie.....	26
IV. Transposition didactique :	27
Les deux étapes de la transposition didactique	27
V. Trame conceptuelle.....	28
VI. Niveau de formulation d'un concept :.....	29
VII. Les erreurs	30
VIII. Les représentations/conceptions initiales	31
1. Définition.....	31
2. Émergence et évolution des représentations :.....	32
IX. Obstacles.....	33
X. Objectif-obstacle :.....	34
XI. Conflit sociocognitif :.....	34
XII. Modèles didactiques	35
Théorie de la situation didactique.....	37
I. Introduction.....	37
II. Contrat didactique :.....	38
III. CONTRAT DIDACTIQUE : TOPOGENÈSE ET CHRONOGENÈSE.....	38
IV. Situation didactique :	39
V. Situation-problème.....	40
VI. Variables didactiques :.....	40
VII. Ressources didactiques pour l'enseignement et l'apprentissage	41

Histoire et épistémologie de l'informatique

Dans ce cours, nous allons explorer l'histoire et l'épistémologie de l'informatique. L'épistémologie est la branche de la philosophie qui étudie la nature de la connaissance. Elle s'intéresse à la façon dont nous savons ce que nous savons, et aux fondements de la connaissance.

I. Histoire de l'informatique :

L'histoire de l'informatique est une saga fascinante qui remonte à des milliers d'années, mais nous nous concentrerons ici sur les événements clés qui ont façonné le monde de l'informatique moderne.

- **Les débuts de la calculatrice :** Les premières formes de calcul remontent à l'Antiquité, avec des dispositifs rudimentaires comme l'abaque utilisé par les Babyloniens et les bouliers chinois. Ces outils ont évolué au fil des siècles, aboutissant à des calculatrices mécaniques sophistiquées au 17ème siècle, comme la machine à calculer de Pascal et la machine de Leibniz.
- **L'ère des machines à calculer automatiques :** Le XIXe siècle a vu l'émergence de machines à calculer automatiques alimentées par des moteurs, telles que la machine analytique de Charles Babbage, considérée comme le précurseur de l'ordinateur moderne. Bien que ses conceptions n'aient pas été réalisées de son vivant, Babbage a posé les bases théoriques de l'informatique.
- **L'invention du premier ordinateur électronique :** Au cours de la Seconde Guerre mondiale, les besoins militaires ont conduit au développement du premier ordinateur électronique programmable, l'ENIAC (Electronic Numerical Integrator and Computer), achevé en 1946 par J. Presper Eckert et John Mauchly. L'ENIAC était immense, occupant une pièce entière, mais il a ouvert la voie à une ère de calcul électronique.
- **L'avènement des ordinateurs personnels :** Les années 1970 ont vu l'émergence des premiers ordinateurs personnels, avec des entreprises comme Apple, IBM et Commodore lançant des machines révolutionnaires telles que l'Apple II et le Commodore PET. Ces ordinateurs ont rendu la technologie informatique accessible aux particuliers et ont ouvert la voie à l'ère de l'informatique domestique.
- **L'explosion d'Internet et du World Wide Web :** Dans les années 1980 et 1990, l'avènement d'Internet et du World Wide Web a radicalement transformé le paysage informatique. Tim Berners-Lee a inventé le WWW en 1989, ouvrant la porte à une connectivité mondiale sans précédent. Internet est devenu un outil omniprésent, transformant la façon dont nous communiquons, travaillons et vivons.
- **L'avènement de l'informatique moderne :** Au cours des dernières décennies, l'informatique a connu une croissance exponentielle, avec des avancées telles que l'intelligence artificielle, l'informatique quantique et l'Internet des objets. Ces progrès continuent de repousser les frontières de ce qui est possible, ouvrant la voie à un avenir où la technologie informatique continue de façonner notre monde de manière profonde et significative.

II. L'épistémologie de l'informatique

1. L'épistémologie : un outil de compréhension de la discipline

L'épistémologie est la branche de la philosophie qui étudie la nature, les méthodes et la validité de la connaissance scientifique. Appliquée à l'informatique, elle permet de répondre à des questions fondamentales que tout enseignant devrait se poser :

- **Ce qu'est l'informatique comme discipline**

L'informatique est-elle une science, une technologie, un art, ou un mélange des trois ? Cette question n'est pas qu'académique : elle détermine comment nous l'enseignons. Si nous la percevons uniquement comme une technologie (utilisation d'outils), nous privilégierons la formation à des logiciels. Si nous la considérons comme une science (étude de phénomènes et résolution de problèmes), nous mettrons l'accent sur la modélisation, les algorithmes et la pensée computationnelle. Si nous y voyons aussi une dimension créative et humaine, nous intégrerons des projets, la collaboration et la réflexion éthique.

L'informatique possède en réalité ces trois dimensions simultanément : c'est une science fondamentale (algorithmique, théorie de l'information, calculabilité), une ingénierie (conception de systèmes, programmation, architecture), et une pratique sociale et créative (impact sur la société, création d'applications, résolution de problèmes concrets). Comprendre cette multi-dimensionnalité est essentiel pour enseigner la discipline dans sa richesse.

- **La nature des connaissances produites**

L'informatique ne produit pas le même type de connaissances que les mathématiques, la physique ou les sciences humaines. Elle produit :

- Des concepts abstraits (algorithme, information, complexité, abstraction, récursivité)
- Des artefacts concrets (logiciels, systèmes, protocoles, applications)
- Des méthodes de résolution de problèmes (décomposition, reconnaissance de motifs, généralisation)
- Des modèles de la réalité (simulation, bases de données, intelligence artificielle)
- Des normes et conventions sociales (formats de fichiers, protocoles de communication, licences open source)

Reconnaître cette diversité aide l'enseignant à varier les modes d'enseignement et d'évaluation. On n'enseigne pas un concept abstrait comme on enseigne l'utilisation d'un logiciel, et on n'évalue pas la créativité algorithmique comme on évalue la maîtrise d'une syntaxe.

- **Comment ces connaissances sont construites et validées**

En informatique, la validation d'une connaissance prend plusieurs formes :

- La preuve formelle mathématique (pour les algorithmes, la complexité, la correction)
- L'expérimentation et le test (pour les programmes, les systèmes)
- L'usage et l'adoption sociale (pour les standards, les outils, les pratiques)
- Le débat et la critique (pour les questions éthiques, l'architecture des systèmes)

Comprendre ces modes de validation permet à l'enseignant de proposer des situations d'apprentissage authentiques : faire prouver un algorithme, faire tester un programme, faire débattre d'un choix de conception, faire analyser l'impact social d'une technologie. Chacune de ces activités correspond à une manière légitime de produire et valider des connaissances en informatique.

2. Les principales approches épistémologiques de l'informatique

Différentes manières de concevoir l'informatique coexistent, chacune mettant l'accent sur une dimension particulière de la discipline. Comprendre ces approches permet à l'enseignant de diversifier ses pratiques et d'éviter une vision réductrice.

L'approche mécaniste (ou techno-centrée)

Cette approche met l'ordinateur au centre : l'informatique est vue comme l'étude du fonctionnement des machines et de l'exécution des programmes.

Principes clés :

- L'ordinateur est une machine qui exécute des instructions de manière déterministe
- Comprendre l'informatique, c'est comprendre comment la machine fonctionne

- L'accent est mis sur la syntaxe, l'exécution pas à pas, les mécanismes internes

Points forts :

- Rigueur et précision dans la compréhension des mécanismes
- Compréhension fine du fonctionnement de la machine
- Base solide pour optimiser les programmes

Limites :

- Risque de réduire l'informatique à l'exécution mécanique
- Peu d'accent sur la résolution de problèmes et la créativité
- Peut démotiver les élèves par un excès de technicité

L'approche cognitive (ou centrée sur la résolution de problèmes)

Cette approche considère l'informatique comme une science de la résolution de problèmes et de la modélisation.

Principes clés :

- L'informatique développe des modes de pensée spécifiques (décomposition, abstraction, algorithmes)
- Programmer, c'est avant tout résoudre des problèmes
- L'ordinateur est un outil au service de la pensée, pas une fin en soi

Points forts :

- Développe l'autonomie et la capacité à transférer
- Met l'accent sur la compréhension conceptuelle
- Favorise la créativité et l'innovation

Limites :

- Peut négliger les aspects techniques nécessaires
- Risque d'abstraction excessive pour certains élèves
- Nécessite du temps pour la réflexion

L'approche constructiviste (ou centrée sur l'apprentissage)

Cette approche s'intéresse à comment les élèves construisent leurs connaissances en informatique, souvent par essais-erreurs et confrontation à des obstacles.

Principes clés :

- L'apprentissage se fait par construction progressive, pas par transmission
- Les erreurs et les obstacles sont des occasions d'apprendre
- L'élève doit être acteur de son apprentissage

Points forts :

- Apprentissage durable et significatif
- Développe l'autonomie et la métacognition
- Respecte les rythmes individuels

Limites :

- Peut être chronophage
- Nécessite un guidage fin de l'enseignant
- Tous les concepts ne se prêtent pas à la découverte

L'approche socio-constructiviste (ou collaborative)

Cette approche considère que l'apprentissage se fait par interaction sociale et construction collective du savoir.

Principes clés :

- On apprend mieux en collaborant avec d'autres

- Le conflit socio-cognitif (confrontation d'idées) fait progresser
- Le langage et la verbalisation sont essentiels

Points forts :

- Développe des compétences transversales (communication, travail d'équipe)
- Reflète les pratiques professionnelles réelles
- Motivation par l'entraide et l'émulation

Limites :

- Nécessite une bonne gestion de classe
- Risque de déséquilibres dans les groupes
- Évaluation individuelle plus complexe

L'approche socio-culturelle (ou critique)

Cette approche considère l'informatique comme une pratique sociale, culturelle et politique, avec des enjeux éthiques et citoyens.

Principes clés :

- L'informatique n'est pas neutre, elle transforme la société
- Les choix technologiques ont des conséquences sociales et politiques
- Former en informatique, c'est aussi former des citoyens critiques

Points forts :

- Développe l'esprit critique et la responsabilité
- Contextualise les apprentissages
- Donne du sens à la discipline

Limites :

- Peut être perçue comme éloignée de la technique
- Nécessite une culture générale solide de l'enseignant
- Débats parfois difficiles à gérer

L'approche créative (ou centrée sur la production)

Cette approche voit l'informatique comme un espace de création et d'expression, proche des arts ou du design.

Principes clés :

- Programmer, c'est créer quelque chose de nouveau
- La créativité est au cœur de l'informatique
- L'accent est mis sur les projets, l'esthétique, l'expérience utilisateur

Points forts :

- Grande motivation des élèves
- Développe l'imagination et l'initiative
- Valorise les productions personnelles

Limites :

- Risque de négliger les fondamentaux
- Peut favoriser les élèves déjà créatifs
- Évaluation parfois subjective

3. Pour l'enseignant : des implications pratiques directes

Éclairer les choix pédagogiques

Sans réflexion épistémologique, l'enseignant risque de reproduire inconsciemment la manière dont il a lui-même appris l'informatique, ou de céder aux pressions utilitaristes (enseigner uniquement ce qui est "utilisable" immédiatement). L'épistémologie offre un cadre pour faire des choix conscients et justifiés.

Par exemple :

- **Si j'adopte une approche cognitive** (résolution de problèmes), je commencerai mes séances par des problèmes ouverts à résoudre, puis j'introduirai les concepts et outils nécessaires. Je poserai des questions comme : "Comment feriez-vous pour trier ces cartes le plus rapidement possible ?" avant d'introduire les algorithmes de tri.
- **Si j'adopte une approche constructiviste**, je proposerai des activités où les élèves découvrent par eux-mêmes les limites d'une approche naïve avant d'introduire une solution plus sophistiquée. Par exemple, je leur ferai d'abord coder une recherche séquentielle dans une liste, constater sa lenteur sur de grandes données, puis découvrir l'intérêt de la recherche dichotomique.
- **Si j'adopte une approche socio-culturelle**, j'organiserai des projets collaboratifs, des débats sur les usages des technologies, et j'inviterai des professionnels pour témoigner. Je ferai analyser des cas concrets : "Cette application de reconnaissance faciale, quels avantages ? Quels risques pour les libertés ?"
- **Si j'adopte une approche créative**, je proposerai des projets libres où les élèves conçoivent leurs propres applications ou jeux, en valorisant l'originalité et l'esthétique autant que la correction technique.
- **Si j'adopte une approche mécaniste**, je consacrerai du temps à décortiquer comment fonctionne un programme, à tracer l'exécution pas à pas, à comprendre ce qui se passe en mémoire.
- Ces choix ne sont pas arbitraires : ils découlent d'une vision épistémologique de ce qu'est l'informatique et de comment on apprend en informatique. L'important est de ne pas s'enfermer dans une seule approche, mais de les mobiliser selon les objectifs, les concepts enseignés et les profils des élèves.

Guider les types d'activités proposées

Chaque approche épistémologique privilégie certains types d'activités :

Activités mécanistes :

- Exécution pas à pas d'algorithmes sur papier
- Analyse du fonctionnement interne d'un système
- Étude de la représentation des données en mémoire
- Débogage méthodique avec suivi de variables
- Exemple : "Tracez l'exécution de cet algorithme de tri sur le tableau [5, 2, 8, 1, 9]"

Activités cognitives :

- Résolution de problèmes algorithmiques sans ordinateur
- Conception d'algorithmes avant toute programmation
- Comparaison de solutions alternatives et analyse de leur efficacité
- Généralisation à partir de cas particuliers
- Exemple : "Trouvez une stratégie pour ranger ces livres par ordre alphabétique en un minimum d'opérations"

Activités constructivistes :

- Situations-problèmes avec obstacles cognitifs
- Phases d'exploration libre suivies de mises en commun
- Carnets de bord pour suivre l'évolution de la compréhension
- Moments de métacognition : "Qu'est-ce qui était difficile ? Pourquoi ?"
- Exemple : "Essayez de coder ce jeu. Quelles difficultés rencontrez-vous ? Comment les surmonter ?"

Activités socio-constructivistes :

- Projets de groupe avec répartition des tâches
- Pair programming (programmation en binôme)
- Revue de code par les pairs
- Débats argumentés sur des choix de conception
- Exemple : "En binôme, l'un code pendant que l'autre guide. Alternez toutes les 10 minutes."

Activités socio-culturelles :

- Analyse critique d'applications (vie privée, biais, accessibilité)
- Études de cas sur des innovations et leurs conséquences
- Débats sur des dilemmes éthiques
- Rencontres avec des professionnels
- Exemple : "Analysez cette application de livraison : qui gagne, qui perd ? Quels algorithmes décident ?"

Activités créatives :

- Projets libres avec cahier des charges minimal
- Hackathons ou concours de programmation créative
- Création de jeux, d'animations, d'art génératif
- Présentation et évaluation par les pairs
- Exemple : "Créez une application qui aide les lycéens dans leur quotidien. Soyez originaux !"

Un enseignant conscient de ces différentes dimensions veillera à varier les types d'activités pour offrir une formation complète et équilibrée, en combinant souvent plusieurs approches dans une même séquence.

Définir la posture professionnelle en classe

- L'épistémologie influence aussi la manière dont l'enseignant se positionne face à ses élèves et face au savoir :
- **L'enseignant transmetteur** (approche mécaniste) : Il détient le savoir, le structure et le transmet de manière claire et progressive. Il montre, explique, corrige. Cette posture a du sens quand il s'agit d'enseigner des concepts stabilisés et des fondamentaux (syntaxe, concepts de base).
- **L'enseignant facilitateur** (approche cognitive) : Il pose des défis, questionne, guide la réflexion sans donner la solution immédiatement. Il dit : "Essayez d'abord", "Comment pourriez-vous procéder ?", "Quelle autre solution existe ?" Cette posture favorise l'autonomie et la construction de stratégies.
- **L'enseignant accompagnateur** (approche constructiviste) : Il observe les élèves travailler, identifie leurs obstacles, propose des aides ciblées au bon moment. Il accepte les erreurs comme des étapes normales. Il dit : "Qu'avez-vous essayé ?", "Pourquoi ça ne marche pas selon vous ?"
- **L'enseignant médiateur** (approche socio-constructiviste) : Il organise les interactions, facilite les échanges, arbitre les désaccords. Il valorise les explications entre pairs. Il dit : "Expliquez votre idée au groupe", "Qu'en pensez-vous les autres ?"
- **L'enseignant citoyen critique** (approche socio-culturelle) : Il aide les élèves à prendre du recul sur les technologies, à questionner leurs usages, à développer un esprit critique. Il dit : "Qui a intérêt à ce que cette technologie existe ?", "Quelles conséquences pour la société ?"
- **L'enseignant coach créatif** (approche créative) : Il encourage, stimule l'imagination, valorise les idées originales même imparfaites. Il dit : "Et si vous essayiez autrement ?", "C'est audacieux, j'aime bien !"
- Un enseignant qui a réfléchi à l'épistémologie de sa discipline sait jongler entre ces postures selon les moments et les objectifs pédagogiques, plutôt que de s'enfermer dans une seule manière d'enseigner.

4. Au-delà de la technique : former des enseignants réflexifs

Parler d'épistémologie c'est aussi inviter les enseignants à devenir des praticiens réflexifs capables de :

- **Justifier leurs choix pédagogiques** : Pourquoi ai-je choisi de commencer par le code plutôt que par un problème papier-crayon ? Pourquoi ai-je privilégié le travail individuel sur cet exercice ? Ces choix doivent pouvoir être explicités et discutés, pas seulement suivis par habitude ou imitation.

- **Analyser leurs pratiques** : Est-ce que mes séances mobilisent toujours la même approche épistémologique ? Est-ce que je laisse de la place à la créativité des élèves ou est-ce que je leur impose toujours une solution unique ? Est-ce que je favorise toujours les mêmes types d'élèves (ceux qui aiment la technique, ceux qui sont créatifs, etc.) ? Cette analyse permet de faire évoluer sa pratique.
- **S'adapter aux différents profils d'élèves** : Certains élèves apprennent mieux par la manipulation et l'expérimentation (approche constructiviste), d'autres par la compréhension rigoureuse des mécanismes (approche mécaniste), d'autres par des projets motivants (approche créative), d'autres par la collaboration (approche socio-constructiviste). Un enseignant conscient de la diversité épistémologique peut diversifier ses approches pour toucher tous les élèves.
- **Se positionner professionnellement** : Face à un collègue qui réduit l'informatique à l'usage d'outils bureautiques, face à une direction qui veut "moderniser" en achetant des tablettes sans réfléchir à la pédagogie, face à des parents qui veulent que leurs enfants "apprennent à coder" sans comprendre ce que cela implique, l'enseignant doit pouvoir défendre une vision argumentée, nuancée et ambitieuse de sa discipline.
- **Continuer à apprendre et à évoluer** : L'informatique change rapidement (nouveaux langages, nouveaux outils, nouveaux enjeux comme l'intelligence artificielle). Un enseignant qui comprend la structure profonde de sa discipline (son épistémologie) peut intégrer ces nouveautés sans se perdre, en gardant le cap sur l'essentiel : développer la pensée informatique, la créativité, la rigueur et l'esprit critique de ses élèves, quelle que soit la technologie du moment.

5. Un enjeu de légitimité de la discipline

Enfin, parler d'épistémologie, c'est contribuer à la légitimation de l'informatique comme discipline scolaire à part entière.

Trop souvent, l'informatique est encore perçue comme :

- Un simple ensemble d'outils au service d'autres disciplines
- Une compétence technique sans profondeur conceptuelle
- Une mode passagère liée aux technologies du moment
- Quelque chose que "tout le monde peut apprendre tout seul sur YouTube"

En développant une réflexion épistémologique, les enseignants d'informatique peuvent montrer que :

- L'informatique possède des concepts fondamentaux aussi riches et exigeants que ceux des mathématiques ou de la physique
- Elle développe des modes de pensée spécifiques et transférables au-delà de la programmation
- Elle pose des questions philosophiques, éthiques et sociales majeures pour notre temps
- Elle nécessite une pédagogie réfléchie et adaptée, pas seulement une formation technique ou une simple familiarisation avec des outils

C'est en formant des enseignants capables de penser et d'explicitier l'épistémologie de leur discipline qu'on construira une informatique scolaire ambitieuse, exigeante et formatrice pour tous les élèves, qui ne se réduit ni à l'usage d'outils, ni à un apprentissage mécanique du code.

En résumé : L'épistémologie n'est pas un luxe théorique réservé aux chercheurs en didactique. C'est un outil indispensable pour tout enseignant d'informatique qui veut :

- Comprendre en profondeur ce qu'il enseigne
- Faire des choix pédagogiques éclairés et justifiés
- Diversifier ses pratiques pour toucher tous les élèves
- Former non seulement des codeurs, mais des penseurs, des créateurs et des citoyens critiques
- Défendre et légitimer sa discipline dans l'institution scolaire
- Continuer à évoluer professionnellement tout au long de sa carrière

III. Filiation conceptuel de l'informatique

l'informatique a une double filiation : une filiation conceptuelle d'un côté avec une discipline et une filiation matériel ,technologique avec une autre discipline

l'informatique est conceptuellement fille des mathématiques en ce sens que beaucoup de modèles de calcul qui ont été intégrés en informatique et qui ont fait son corpus scientifique sont des modèles qui ont été mises en place au départ par des mathématiciens dans le cadre des mathématiques. En effet, l'informatique en gros dans ses modèles et dans ce qu'elle propose est issu deux champs disciplinaires des mathématiques : l'algèbre et la logique . et de ce fait là, sa manière de procéder et de percevoir le monde est très liée à ces deux domaines.

elle est aussi technologiquement , sur le plan des méthodes et des outils la fille de l'électronique numérique, puisque l'informatique ne peut pas se passer d'ordinateur et que par conséquent ces ordinateurs eux-mêmes sont les produits de l'électronique numérique.

et cette double filiation fait que l'informatique est hybride .

les informaticiens ,donc, ont une leur double appartenance. ils ont une appartenance d'un côté au monde des idées des connaissances de la réflexion de la philosophie, et de l'autre côté à la matière ,aux circuits, aux objets et à tous ces produits qui relève de l'industrie.

1. la relation croisée entre l'informatique et les autres sciences :

on parle de relation croisée lorsque non seulement il y a une **importation** possible d'autres disciplines, mais il y a aussi une modification des notions et des pratiques qui viennent de ces disciplines, et également il y a le fait de donner à d'autres disciplines des modèles qui sont issus la discipline elle-même . donc ,il s'agit d' un véritable échange et pas simplement une situation hiérarchique de domination de discipline vers d'autres. puisque l'informatique a une composante scientifique et une nature technologique , alors on a deux niveaux :le plan des modèles qui relèverait de la nature scientifique de l'informatique et le plan des techniques qui relèverait de la nature technologique de l'informatique .

En informatique, il existe de nombreux modèles partagés entre d'autres disciplines qui concernent aussi bien les sciences de l'homme et de la société que les sciences du langage, la philosophie , l'art, la biologie, la physique, l'économie...etc. donc , il existe des sous domaines de l'informatique qui sont inspirés d'autres sciences :

- ***l'intelligence artificielle*** avec toutes ses représentations de la mémoire de l'esprit du fonctionnement des unités de calcul les processeurs etc a une terminologie commune avec ***la psychologie cognitive*** .elle est également en relation fondamentale avec la neuroscience. En effet, on parle aujourd'hui de réseaux de neurones formelle en fait que fait-on d'autre que de tenter de modéliser via des graphes extrêmement connexes des structures naturelles qui sont étudiées par les neurosciences et inversement les neurosciences sont intéressés par les simulations qui sont faites dans les modèles de perception et de commissions qui sont proposés en vision artificielle par exemple
- ***les systèmes d'information*** :sont inspirés ***des sciences de gestion***
- ***les modèles de l'interaction homme/machine*** sont impactés par ***l'ergonomie et les sciences du travail***
- ***les systèmes multi agents*** se sont inspirés de ***la sociologie et la psychologie sociale***
- ***les langages de programmation*** donnent à l'informatique une dimension langagière extrêmement importante . de ce fait là , elle partage avec ***la linguistique formelle*** d'une parenté très grande.
- ***Le traitement automatique des langues*** (TALN ,PLN) ont également une relation étroite avec les ***sciences de langage***
- ***la cryptographie*** : s'est inspiré des sciences du code et du codage.

- **la bio-informatique** : La bio-informatique est une science à l'interface de l'informatique et des sciences de la vie (biochimie, biologie, microbiologie, écologie, épidémiologie). Étant donné que les scientifiques de la vie génèrent une quantité croissante de nouvelles données portant sur les génomes, les biomolécules, les organismes, leurs interactions et leur évolution, il y a un besoin croissant d'approches informatiques pour la manipulation, le stockage, la visualisation et l'analyse de ces données souvent très complexes.

L'informatique a été utilisée par les sciences sur le plan de la technique ou de la technologie en tant qu'outil et service. Ces sciences utilisent l'informatique comme mode de calcul sur leurs données, et également comme système de résolution de problèmes à grande échelle. L'informatique est également utilisée comme un environnement de simulation qui est sans danger et qui remplace la réalité par la virtualité pour pouvoir éviter les éventuels aspects nocifs ou néfastes des expérimentations *in vivo*.

2. Question de l'identité de l'informatique

Les sous-domaines qui résultent de l'interaction de l'informatique avec d'autres disciplines font émerger la question de l'identité de l'informatique : qui est l'informaticien ?

On peut dire qu'un chercheur dans un domaine bi ou pluridisciplinaire est un informaticien si son objectif est d'améliorer la science informatique en termes de modèles de concepts ou de méthode en proposant un meilleur algorithme ou un meilleur modèle plus efficace, plus performants, moins coûteux et plus couvrants.

D'autre part, on peut dire qu'un ingénieur dans un domaine bi ou pluridisciplinaire est un informaticien si son objectif est d'améliorer la technologie informatique en termes de pratiques des expertises et de formes applicatives nouvelles.

3. Dimensions de l'informatique

L'informatique est une discipline complexe qui englobe plusieurs dimensions, chacune jouant un rôle crucial dans son développement et son application. Voici quelques-unes des dimensions clés de l'informatique :

1. **Dimension scientifique** : Cette dimension concerne la recherche fondamentale en informatique. Elle comprend l'étude des principes théoriques sous-jacents à la computation, tels que la théorie de la complexité, l'algorithmique, la théorie des langages formels et la logique mathématique. La dimension scientifique de l'informatique vise à comprendre les limites de ce qui peut être calculé et à développer de nouvelles techniques et théories pour résoudre des problèmes complexes.
2. **Dimension technique** : La dimension technique de l'informatique concerne l'application des principes scientifiques à la conception, au développement et à la mise en œuvre de systèmes informatiques concrets. Cela inclut le développement de logiciels, la conception de systèmes matériels, la gestion de réseaux informatiques et la sécurité informatique. Cette dimension met l'accent sur la maîtrise des outils et des technologies nécessaires pour créer des solutions informatiques fonctionnelles et efficaces.
3. **Dimension sociale et humaine** : L'informatique a un impact profond sur la société et les individus, ce qui donne naissance à une dimension sociale et humaine. Cela comprend l'étude de l'interaction entre la technologie informatique et les individus, les communautés et la société dans son ensemble. Cela englobe des sujets tels que l'éthique informatique, la vie privée des données, l'informatique inclusive et l'impact socio-économique de la technologie.
4. **Dimension économique** : L'informatique est un moteur majeur de l'économie mondiale, ce qui donne lieu à une dimension économique importante. Cela inclut l'étude des modèles d'affaires liés à la technologie informatique, l'innovation technologique en tant que catalyseur de la croissance économique, et les questions de propriété intellectuelle et de commercialisation des produits informatiques.
5. **Dimension éthique** : Avec le pouvoir croissant de la technologie informatique vient la responsabilité éthique de son utilisation. La dimension éthique de l'informatique concerne l'examen des implications morales et sociales de la technologie informatique, ainsi que la promotion de pratiques responsables en matière de conception, de développement et d'utilisation des systèmes informatiques.

Ensemble, ces différentes dimensions de l'informatique contribuent à une compréhension holistique de la discipline et de son impact sur le monde moderne.

IV. Concepts de base de l'informatique:

L'informatique, en tant que discipline scientifique, s'est construite autour de quatre concepts fondamentaux qui structurent à la fois la recherche, les pratiques professionnelles et l'enseignement : la machine, l'algorithme, le langage et l'information. Ces concepts ne sont pas arbitraires. Ils émergent de l'histoire de la discipline et répondent à des questions essentielles :

1. **Machine** : Qu'est-ce qui peut calculer ? Quelles sont les limites du calcul automatique ?
 - En informatique, une machine fait référence à tout dispositif physique ou virtuel capable d'exécuter des opérations informatiques. Cela peut inclure des ordinateurs personnels, des serveurs, des smartphones, des tablettes, des objets connectés, etc.
2. **Algorithme** : Comment résoudre un problème de manière systématique et reproductible ?
 - Un algorithme est une séquence finie et non ambiguë d'instructions ou de règles permettant de résoudre un problème ou d'effectuer une tâche spécifique. Les algorithmes sont utilisés pour décrire la logique ou les étapes nécessaires à l'exécution d'une opération.
 - Les algorithmes sont essentiels en informatique pour la conception de programmes et de systèmes informatiques. Ils peuvent être exprimés sous forme de pseudo-code, de diagrammes de flux ou de langages de programmation.
3. **Langage** : Comment exprimer et communiquer des procédures de calcul ?
 - En informatique, un langage est un système formel de communication utilisé pour écrire des programmes informatiques, décrire des algorithmes, et communiquer avec des machines. Les langages de programmation peuvent être de différents types, notamment les langages de haut niveau comme Python, Java, C++, etc., et les langages de bas niveau comme le langage machine et l'assembleur.
 - Chaque langage de programmation a sa propre syntaxe, sa sémantique et ses règles spécifiques qui permettent aux programmeurs d'exprimer des instructions de manière précise et efficace pour résoudre des problèmes informatiques.
4. **Information** : Qu'est-ce qui est traité, transformé, transmis par les machines ?
 - L'information en informatique fait référence aux données traitées, organisées et interprétées de manière significative. Cela peut inclure des données textuelles, des images, des sons, des vidéos, etc.
 - Les données brutes deviennent de l'information lorsqu'elles sont analysées et interprétées dans un contexte spécifique pour fournir une signification ou une utilité. Par exemple, une série de chiffres peut représenter des données brutes, mais une fois traitée et analysée, elle peut fournir des informations telles que des tendances, des statistiques ou des prévisions.

V. La dualité d'utilisation de l'informatique : Objet d'enseignement et outil pédagogique

L'informatique occupe une place particulière dans l'éducation, car elle est à la fois un **objet d'enseignement** et un **outil pédagogique**. Cette dualité soulève des questions importantes sur la manière dont l'informatique doit être intégrée dans les programmes scolaires et utilisée pour maximiser l'apprentissage des élèves.

○ En tant qu'objet d'enseignement:

Enseigner l'informatique comme discipline à part entière, c'est :

- Transmettre des concepts fondamentaux : algorithme, abstraction, machine, langage, information
- Développer des compétences spécifiques : pensée computationnelle, programmation, modélisation
- Former à des méthodes de travail : décomposition de problèmes, débogage, conception rigoureuse
- Initier à une culture scientifique et technique : histoire de l'informatique, enjeux sociétaux

○ En tant qu'outil pédagogique:

Utiliser l'informatique **au service d'autres apprentissages**, c'est :

- Employer des logiciels éducatifs pour enseigner les maths, la physique, l'histoire...
- Créer des supports numériques interactifs (présentations, vidéos, quiz...)
- Organiser des activités collaboratives via des plateformes en ligne
- Différencier les apprentissages grâce au numérique (exercices adaptatifs, ressources variées)

La dualité objet/outil : une tension à transformer en richesse

Le dilemme professionnel

L'enseignant d'informatique vit une tension constante entre deux rôles : enseigner l'informatique comme discipline scientifique avec ses concepts propres (algorithmes, structures de données, pensée computationnelle) ou la mobiliser comme outil au service d'autres disciplines (simulations en physique, visualisation de données en SVT, recherche documentaire en histoire). Cette dualité crée des situations paradoxales : si l'informatique est réduite à un rôle d'outil, elle devient invisible et l'enseignant est perçu comme "le technicien qui dépanne" ; si elle est enseignée uniquement comme discipline abstraite, les élèves ne voient pas son utilité concrète et se démotivent. Le risque majeur est l'instrumentalisation : l'informatique perd sa légitimité disciplinaire et devient une simple "compétence transversale" sans profondeur conceptuelle.

Dépasser la dualité par l'articulation consciente

Plutôt que de choisir entre ces deux dimensions, l'enseignant peut les articuler en appliquant deux principes complémentaires.

- **Premier principe** : expliciter systématiquement les concepts informatiques mobilisés, même en mode "outil". Par exemple, dans un projet SVT sur la pollution, au lieu de simplement "montrer comment faire un graphique dans Excel", l'enseignant nomme et enseigne les concepts : structuration de données, algorithmes de filtrage, visualisation, automatisation.
- **Deuxième principe** : contextualiser les concepts "disciplinaires" dans des usages réels. Ainsi, l'enseignement des algorithmes de tri ne se limite pas à la théorie pour l'examen, mais montre leurs applications concrètes (moteurs de recherche, réseaux sociaux, e-commerce). Cette approche intégrée nécessite de négocier sa place dans les projets interdisciplinaires dès le départ, en clarifiant les objectifs d'apprentissage en informatique et les modalités d'évaluation, transformant ainsi la tension initiale en richesse pédagogique.

VI. Modes de pensée en informatique :

Introduction : Au-delà de la technique, des façons de penser

L'informatique ne se réduit pas à un ensemble de **compétences techniques** (savoir coder, utiliser un logiciel). Elle développe des **modes de pensée** spécifiques, c'est-à-dire des manières d'appréhender et de résoudre des problèmes qui peuvent s'appliquer bien au-delà de l'informatique elle-même.

Question centrale : Qu'est-ce qu'un élève développe comme compétences cognitives en apprenant l'informatique, au-delà de savoir écrire du code Python ?

La pensée logique : raisonner avec rigueur

Définition et applications en informatique

La pensée logique est la capacité à :

- raisonner de manière déductive (si A implique B et A est vrai, alors B est vrai), inductive (observer des cas particuliers pour en tirer une règle générale),
- manipuler des propositions (ET, OU, NON, SI...ALORS...)
- identifier des contradictions ou incohérences.

En informatique, elle se manifeste concrètement dans plusieurs contextes : **en programmation** à travers les conditions (if $x > 0$ and $y < 10$), les boucles conditionnelles (while not fin) et l'algèbre booléenne (True, False, and, or, not) ; **en bases de données** via les requêtes SQL (SELECT * FROM élèves WHERE age $\geq$ 16 AND ville = 'Casablanca') ; **en électronique numérique** avec les portes logiques (AND, OR, NOT, XOR) et les circuits combinatoires. Cette pensée assure la rigueur et la précision du raisonnement informatique.

Place dans l'enseignement et limites

La pensée logique est nécessaire mais insuffisante pour l'informatique. Elle n'est pas spécifique à la discipline : elle est commune aux mathématiques et à la philosophie. Un élève peut exceller en logique formelle (tables de vérité, syllogismes) mais peiner à décomposer un problème complexe, à abstraire l'essentiel ou à concevoir un algorithme créatif.

Les activités recommandées incluent les exercices de logique (tables de vérité), les énigmes logiques (type "Einstein's riddle"), l'analyse de conditions complexes dans des programmes et le debugging d'erreurs logiques (conditions inversées, négations oubliées).

La pensée algorithmique : concevoir des procédures efficaces

Définition et composantes

La pensée algorithmique est la capacité à concevoir des séquences d'étapes pour résoudre un problème de manière systématique. Elle implique quatre compétences essentielles :

1. concevoir des procédures – créer une suite d'instructions ordonnées et non ambiguës ;
2. raisonner sur l'efficacité – évaluer si la solution est rapide et optimale (peut-on faire mieux ?) ;
3. prouver la correction – s'assurer que la procédure donne toujours le bon résultat pour tous les cas valides ;
4. anticiper les cas limites – prévoir ce qui se passe si l'entrée est vide, négative, ou très grande.

Cette pensée se distingue de la simple exécution d'instructions : elle exige créativité, anticipation et rigueur.

Implications pour l'enseignement

L'enseignement de la pensée algorithmique doit suivre une progression en quatre niveaux : (1) exécuter un algorithme donné en suivant les instructions pas à pas ; (2) modifier un algorithme existant pour l'adapter à un nouveau problème ; (3) concevoir un algorithme de zéro avec créativité ; (4) comparer plusieurs algorithmes et analyser leur complexité.

Les activités privilégiées incluent les algorithmes sans ordinateur (trier des cartes, chercher dans un dictionnaire), l'écriture de pseudo-codes avant toute programmation, les jeux algorithmiques (Lightbot, Code.org) et l'analyse de complexité ("Que se passe-t-il avec 1000 éléments au lieu de 10 ?"). L'évaluation ne doit pas se limiter à vérifier si le code fonctionne, mais interroger le processus : "Pourquoi avez-vous choisi cette approche ?", "Comment pourriez-vous optimiser ?", "Votre algorithme fonctionne-t-il pour tous les cas ?".

La pensée computationnelle (Computational Thinking)

Définition et piliers fondamentaux

Popularisée par Jeannette Wing (2006), la pensée computationnelle est la capacité à formuler des problèmes et leurs solutions de manière qu'ils puissent être résolus de façon systématique, que ce soit par un ordinateur ou un humain suivant une procédure. Elle repose sur quatre piliers complémentaires :

(1) la **décomposition** – diviser un problème complexe en sous-problèmes gérables (exemple : organiser une fête scolaire en budget, lieu, invitations, activités...) ;

(2) la **reconnaissance de motifs** – identifier des régularités et généraliser (exemple : repérer qu'un tri consiste toujours à comparer et échanger) ;

(3) l'**abstraction** – filtrer l'information pour ne garder que l'essentiel (exemple : un plan de métro simplifie la géographie pour ne garder que l'ordre des stations) ;

(4) la **conception d'algorithmes** – créer une séquence d'étapes précises, générales et efficaces pour résoudre un problème. Ces quatre composantes se renforcent mutuellement et constituent le cœur de la discipline informatique.

Implications pédagogiques

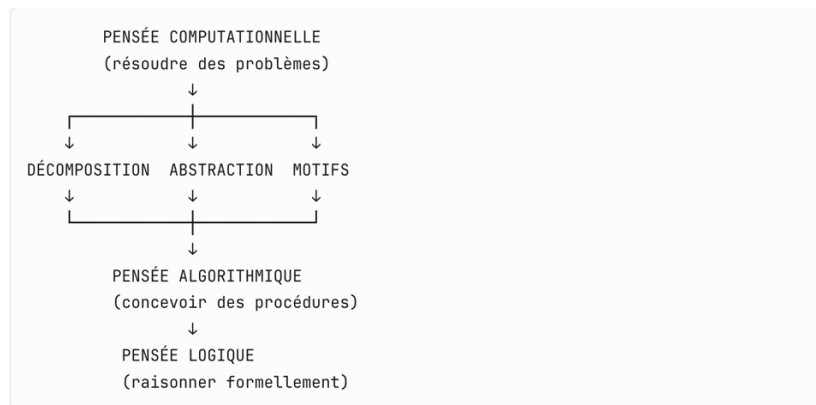
La pensée computationnelle n'est pas réservée à la programmation : elle est transférable à toutes les disciplines (décomposer une révolution en causes multiples en histoire, modéliser une chaîne alimentaire en SVT, planifier un voyage dans la vie quotidienne). Pour l'enseignant d'informatique, l'enjeu n'est donc pas seulement d'enseigner la syntaxe d'un langage, mais de développer ces modes de pensée à travers des activités variées : activités débranchées (trier des cartes sans ordinateur), projets de création (décomposer un jeu en règles, personnages, score...), exercices de modélisation (représenter un feu tricolore en ne gardant que l'essentiel), et surtout en faisant expliciter les stratégies des élèves ("Comment avez-vous procédé ? Qu'est-ce qui est important ?").

L'évaluation doit porter sur le processus de résolution, pas seulement le code final : demander aux élèves d'expliquer leur démarche de décomposition, d'analyser les difficultés rencontrées, de comparer différentes solutions et d'argumenter leur choix. C'est ainsi qu'on forme des penseurs capables de transférer ces compétences au-delà de l'informatique, plutôt que de simples exécutants de code.

Synthèse : former des penseurs, pas des codeurs

Articulation des trois modes de pensée

Ces trois modes de pensée ne sont pas isolés mais complémentaires et hiérarchisés : la pensée logique fournit la rigueur du raisonnement (conditions, déductions), la pensée algorithmique permet de concevoir des procédures efficaces, et la pensée computationnelle les intègre dans une démarche globale de résolution de problèmes (décomposition + motifs + abstraction + algorithmes).



Relation clé : la pensée algorithmique est une composante de la pensée computationnelle, pas un synonyme. Un élève peut maîtriser l'algorithmique (concevoir des procédures) sans savoir décomposer un problème complexe ou abstraire l'essentiel. De même, un élève excellent en logique peut manquer de créativité algorithmique.

Principes pédagogiques opérationnels

Objectifs d'apprentissage : Ne pas formuler "Les élèves sauront utiliser les boucles for en Python" mais "Les élèves seront capables de décomposer un problème répétitif, identifier quand une boucle est nécessaire, concevoir un algorithme itératif, puis l'implémenter en Python" – on cible les modes de pensée, pas la syntaxe.

Évaluation : Privilégier les problèmes ouverts ("Concevez un algorithme pour..."), l'analyse critique ("Trouvez le bug"), la comparaison de solutions et la métacognition ("Expliquez votre démarche"), plutôt que la syntaxe correcte ou la reproduction de modèles.

Différenciation : Les élèves ont des profils variés (logiques mais peu créatifs, créatifs mais peu rigoureux, besoin de manipulation concrète avant abstraction) – varier les approches (débranchées, projets, formalisation) pour toucher tous les profils.

L'enjeu final : former des penseurs capables de décomposer, abstraire, identifier des motifs, concevoir des procédures efficaces, raisonner avec rigueur logique et transférer ces compétences au-delà de l'informatique, plutôt que de simples exécutants de code.

Les théories d'apprentissage

I. Behaviorisme:

- **Approche:** Apprentissage par le conditionnement opérant.

L'approche behavioriste de l'apprentissage se concentre sur l'observation des comportements externes des apprenants et sur la modification de ces comportements par le biais de renforcements positifs et négatifs. Selon le behaviorisme, l'apprentissage est le résultat de l'association entre des stimuli externes et des réponses comportementales.

- **Exemple:**
 - Utiliser un logiciel de programmation par blocs comme Scratch.
 - Récompenser les élèves lorsqu'ils construisent des programmes corrects.
 - Favoriser la répétition et la mémorisation des instructions.
- **Avantages:** Simple à mettre en place, efficace pour l'apprentissage de concepts de base.
- **Inconvénients:** Ne favorise pas la créativité et la pensée critique, peut être répétitif et ennuyeux.

Situation 1 : Enseignement de la programmation de base à des élèves débutants.

Description : L'enseignant fournit des instructions claires et utilise des techniques de **renforcement** pour **encourager les comportements souhaités chez les élèves**. Les élèves sont exposés à des exemples concrets de code et sont encouragés à imiter et à **reproduire ces modèles**. Les exercices pratiques sont structurés de manière à permettre une progression graduelle, avec des **récompenses immédiates** pour les réalisations.

Situation 2: Créer un programme qui fait dessiner un carré à l'écran.

- **Enseignant:**
 - Divise la classe en petits groupes.
 - Distribue à chaque groupe **une fiche d'instructions** décrivant un programme simple à réaliser avec Scratch.
 - Explique les instructions étape par étape.
 - Passe dans les groupes pour observer et aider les élèves.
- **Élèves:**
 - Suivent les instructions pour construire le programme.
 - Testent le programme et corrigent les erreurs.
 - Reçoivent des récompenses (ex: points, badges) pour chaque programme réussi.

II. Cognitivisme:

- **Approche:** Apprentissage par la manipulation de symboles et de règles.

Le cognitivisme est une approche de l'apprentissage qui met l'accent sur les processus mentaux internes tels que la perception, la mémoire, le raisonnement et la résolution de problèmes. Selon le cognitivisme, l'apprentissage se produit par l'acquisition de connaissances, la compréhension des concepts et la résolution de problèmes à travers la manipulation d'informations mentales.

- **Exemple:**

- Apprendre aux élèves les concepts de base de la programmation comme les variables, les boucles et les fonctions.
- Utiliser des diagrammes et des représentations visuelles pour expliquer les concepts.
- Favoriser la résolution de problèmes et la pensée logique.

- **Avantages:** Favorise la compréhension des concepts, permet une progression structurée.
- **Inconvénients:** Peut être trop théorique et abstrait, ne favorise pas toujours la collaboration et la créativité.

Situation 1 : Enseignement de la résolution de problèmes algorithmiques en informatique.

Description : L'enseignant met l'accent sur la compréhension et la manipulation des concepts sous-jacents. Les élèves sont encouragés à réfléchir de manière critique, à analyser les problèmes et à formuler des solutions en utilisant des structures de pensée abstraites. L'enseignant utilise des stratégies d'enseignement telles que la modélisation cognitive, la résolution de problèmes guidée et les discussions en classe pour aider les élèves à développer leur pensée algorithmique.

Situation 2 : Créer un programme qui demande à l'utilisateur son nom et lui souhaite la bienvenue.

- **Enseignant:**

- Explique les concepts de base de la programmation comme les variables, les boucles et les fonctions.
- Utilise des diagrammes et des représentations visuelles pour expliquer les concepts.
- Propose des exercices et des quiz pour évaluer la compréhension des élèves.

- **Élèves:**

- Écotent l'explication de l'enseignant.
- Prennent des notes et posent des questions.
- Réalisent les exercices et les quiz proposés.

III. Constructivisme:

- **Approche:** Apprentissage par la construction active de connaissances.

Le constructivisme est une approche de l'apprentissage qui met l'accent sur le rôle actif de l'apprenant dans la construction de sa propre compréhension et de son savoir à travers des expériences concrètes, des interactions sociales et des réflexions critiques. Selon le constructivisme, l'apprentissage est un processus de construction et de reconstruction des connaissances par l'interaction entre les nouvelles informations et les connaissances préexistantes.

- **Exemple:**

- Organiser des projets de programmation où les élèves doivent créer leurs propres programmes.
- Encourager les élèves à collaborer et à partager leurs idées.
- Favoriser la créativité et l'innovation.
- **Avantages:** Favorise la motivation et l'engagement des élèves, permet de développer des compétences transversales.
- **Inconvénients:** Nécessite plus de temps et de ressources, peut être difficile à mettre en place pour les enseignants non formés.

Situation 1: Création d'un projet collaboratif de développement de logiciel.

Description : L'enseignant encourage les élèves à construire activement leur propre compréhension et leur savoir à travers **des expériences pratiques et des interactions sociales**. Les élèves sont regroupés en équipes et chargés de concevoir, développer et tester un logiciel fonctionnel. L'enseignant agit en tant que **facilitateur**, fournissant des conseils et des ressources au fur et à mesure que les élèves progressent dans leur projet. Les élèves apprennent en construisant et en partageant leurs connaissances avec leurs pairs.

Situation 2: Créer un jeu de labyrinthe où le joueur doit trouver la sortie.

- **Enseignant:**
 - Défi : "Imaginez et créez un jeu simple en utilisant Scratch."
 - Guide les élèves dans la planification et la réalisation de leur projet.
 - Encourage la collaboration et le partage d'idées entre les élèves.
 - Organise une présentation des projets réalisés.
- **Élèves:**
 - Brainstorming pour trouver des idées de jeux.
 - Planifient leur projet en équipe.
 - Développent leur jeu en utilisant Scratch.
 - Testent et déboguent leur jeu.
 - Présentent leur jeu à la classe.

IV. Connectivisme:

- **Approche:** Apprentissage par la création et le partage de connaissances dans un réseau.

Le connectivisme est une approche de l'apprentissage qui met l'accent sur le rôle des réseaux et des technologies de communication dans la facilitation de l'apprentissage distribué et la création de nouvelles connaissances. Selon le connectivisme, l'apprentissage se produit à travers des connexions et des interactions avec des sources de connaissances externes, telles que des personnes, des ressources en ligne et des réseaux sociaux, plutôt que simplement par l'acquisition de connaissances individuelles.

- **Exemple:**
 - Utiliser des plateformes en ligne comme Code.org pour apprendre à programmer.
 - Participer à des forums et des communautés de programmation en ligne.

- Favoriser l'apprentissage par les pairs et le partage de compétences.
- **Avantages:** Favorise l'apprentissage tout au long de la vie, permet de développer des compétences de collaboration et de communication.
- **Inconvénients:** Nécessite un accès à internet et aux technologies, peut être difficile à suivre pour les élèves en difficulté.

Situation 1: Utilisation de forums en ligne pour l'apprentissage collaboratif de la sécurité informatique.

Description : L'enseignant exploite les réseaux sociaux et les technologies de communication en ligne pour favoriser l'apprentissage distribué et la collaboration entre les élèves. Les élèves participent à des discussions en ligne, partagent des ressources et collaborent à des projets de recherche sur des sujets liés à la sécurité informatique. L'enseignant encourage les élèves à élargir leur réseau professionnel en se connectant avec des experts du domaine et en participant à des communautés en ligne spécialisées. Les élèves apprennent en interagissant avec un réseau étendu de pairs et de mentors.

Situation 2: Réaliser un projet de programmation en équipe en utilisant des outils de collaboration en ligne.

- **Enseignant:**
 - Crée un forum en ligne où les élèves peuvent partager leurs questions et leurs idées.
 - Propose des liens vers des ressources en ligne pour apprendre à programmer.
 - Invite des experts en informatique à intervenir dans la classe.
- **Élèves:**
 - Participent au forum en ligne pour poser des questions et obtenir de l'aide.
 - Recherchent des informations et des tutoriels en ligne.
 - Apprennent des autres élèves et des experts en informatique.
 - Contribuent à la création d'une communauté d'apprentissage.

V. La théorie de l'apprentissage multimédia:

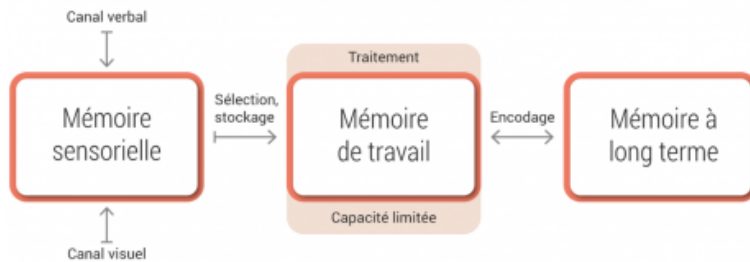
Théorie de la charge cognitive

La théorie de la charge cognitive est un cadre conceptuel qui explore comment la charge mentale imposée lors de l'apprentissage affecte la capacité cognitive de l'individu à traiter les informations. Cette théorie, développée par John Sweller dans les années 1980, met en lumière l'importance de la gestion de la charge cognitive pour optimiser l'apprentissage et la rétention des connaissances.

La théorie de la charge cognitive (TCC) s'intéresse à l'impact de la charge mentale sur l'apprentissage et la performance. Elle stipule que la capacité de traitement de l'information de notre cerveau est limitée, et que la surcharge d'informations peut nuire à l'apprentissage.

- **les principaux concepts de la théorie de la charge cognitive :**

Schéma. Fonctionnement de la mémoire



Source : <https://jenseigneadistance.telug.ca/mod/page/view.php?id=190>

- **Charge cognitive intrinsèque :** Il s'agit de la charge cognitive associée à la complexité inhérente de la tâche ou du matériel d'apprentissage. Les tâches plus complexes ou les concepts plus abstraits imposent une charge cognitive intrinsèque plus élevée, ce qui peut rendre l'apprentissage plus difficile.
- **Charge cognitive extrinsèque :** Il s'agit de la charge cognitive ajoutée par des éléments externes à la tâche principale, tels que des distractions, des informations inutiles ou une présentation peu claire. Une charge cognitive extrinsèque élevée peut surcharger les capacités cognitives de l'individu et entraver l'apprentissage.
- **Charge cognitive germane :** Il s'agit de la charge cognitive qui contribue à la construction de la compréhension et de la rétention des connaissances. Lorsque la charge cognitive germane est optimisée, elle favorise un apprentissage plus profond et plus significatif.
- **Principe de la charge cognitive réduite :** Ce principe préconise la réduction de la charge cognitive extrinsèque pour permettre à l'individu de se concentrer sur la tâche d'apprentissage principale. Cela peut être réalisé en utilisant des présentations claires et concises, en éliminant les distractions et en fournissant des supports pédagogiques appropriés.
- **Principe de la charge cognitive progressive :** Ce principe suggère que les apprenants doivent être exposés progressivement à des tâches ou des concepts plus complexes pour éviter la surcharge cognitive. En introduisant progressivement des éléments plus difficiles, les apprenants ont le temps de s'adapter et de construire leur compréhension de manière incrémentielle.
- **Principe de la charge cognitive extrinsèque équivalente :** Ce principe souligne l'importance de réduire la charge cognitive extrinsèque tout en maintenant un niveau de défi adéquat pour favoriser l'apprentissage. Il s'agit de trouver un équilibre entre la simplification de la présentation et la complexité de la tâche pour maintenir l'engagement et la motivation des apprenants.
- **Effets de la charge cognitive:**
 - **Surcharge cognitive:** lorsque la charge cognitive totale dépasse la capacité de la MT, cela peut entraîner une baisse de l'attention, de la mémoire et de la performance.
 - **Apprentissage inefficace:** l'information n'est pas correctement traitée et stockée en mémoire à long terme.
 - **Difficultés de résolution de problèmes:** la capacité à analyser et à synthétiser l'information est réduite.
- **Applications de la TCC:**
 - Conception pédagogique: réduire la charge cognitive extrinsèque pour optimiser l'apprentissage.
 - Présentation de l'information: utiliser des formats clairs et concis, limiter la quantité d'informations simultanées.
 - Choix des outils pédagogiques: utiliser des technologies et des méthodes adaptées aux besoins des apprenants.
 - Évaluation des apprentissages: tenir compte de la charge cognitive des tâches d'évaluation.

VI. La théorie de l'apprentissage multimédia:

La théorie de l'apprentissage multimédia, également connue sous le nom de théorie de l'apprentissage multimédia intégré (MML), est un cadre théorique qui explore comment les individus acquièrent des connaissances et des compétences à travers l'utilisation de divers médias et technologies. Cette théorie s'appuie sur les principes de l'apprentissage cognitif, en mettant l'accent sur l'interaction entre l'apprenant et le contenu multimédia pour faciliter l'acquisition et la rétention des connaissances.

Voici quelques concepts clés de la théorie de l'apprentissage multimédia :

- **Principe de contiguïté** : Ce principe stipule que les éléments d'apprentissage pertinents doivent être présentés de manière simultanée et spatialement proche dans l'environnement multimédia. Par exemple, un texte accompagné d'images pertinentes peut renforcer la compréhension du sujet présenté.
- **Principe de cohérence** : Ce principe soutient que les éléments multimédias doivent être présentés de manière cohérente et harmonieuse pour éviter la surcharge cognitive et maximiser la compréhension. Les transitions fluides entre les différents médias et les instructions claires contribuent à une expérience d'apprentissage plus efficace.
- **Principe de modalité** : Ce principe suggère que les informations peuvent être présentées de différentes manières (texte, audio, vidéo, animation) en fonction des préférences individuelles des apprenants et de la nature du contenu. Par exemple, certaines personnes peuvent préférer les informations présentées sous forme de vidéo, tandis que d'autres peuvent préférer la lecture de texte.
- **Principe de précharge** : Ce principe propose que les éléments multimédias complexes soient présentés de manière séquentielle et progressive pour éviter la surcharge cognitive. Par exemple, les concepts difficiles peuvent être introduits progressivement avec des explications simples avant de passer à des exemples plus complexes.
- **Principe de signalisation** : Ce principe suggère l'utilisation de signaux visuels ou auditifs pour attirer l'attention des apprenants sur les informations importantes dans le contenu multimédia. Cela peut inclure l'utilisation de couleurs vives, de flèches, de surlignages, ou de sons distinctifs pour indiquer l'importance ou la pertinence d'une information.
- **Principe de personnalisation** : Ce principe encourage la personnalisation du contenu multimédia pour répondre aux besoins individuels des apprenants. Cela peut inclure l'adaptation du niveau de difficulté, le choix des médias préférés, ou la fourniture de rétroaction personnalisée en fonction des performances de l'apprenant.

En combinant ces principes, la théorie de l'apprentissage multimédia vise à concevoir des environnements d'apprentissage efficaces et engageants qui exploitent pleinement les capacités des médias et des technologies pour soutenir l'apprentissage et la rétention des connaissances.

VII. Conclusion:

Le choix de la théorie d'apprentissage la plus appropriée pour l'enseignement de l'informatique dépend de plusieurs facteurs, tels que les objectifs d'apprentissage, le niveau des élèves et les ressources disponibles. Il est important de combiner différentes approches pour créer un environnement d'apprentissage riche et stimulant.

Didactique de l'informatique : concepts généraux

I. Didactique vs pédagogie

1. Didactique

- **Définition:** Du grec ancien διδακτικός, didaktikós (« doué pour l'enseignement »), dérivé du verbe διδάσκω didásko (« enseigner », « instruire »). dans « didactique », l'accent est plutôt porté sur la relation au savoir à transmettre. Contrairement à la pédagogie qui est davantage centrée sur la relation enseignant-élève en vue de l'éducation, la didactique est davantage centrée sur le savoir à transmettre en vue de l'instruction.

La didactique cherche à comprendre comment organiser et structurer les contenus d'enseignement, ainsi que les meilleures approches pour les présenter aux apprenants.

- **Portée :** La didactique se concentre spécifiquement sur les processus d'enseignement et les stratégies utilisées pour transmettre des connaissances dans des domaines spécifiques, tels que les mathématiques, les sciences, la littérature, etc.
- **Approche :** La didactique adopte une approche plus ciblée en se concentrant sur la manière dont les contenus d'enseignement sont structurés, présentés et évalués pour faciliter l'apprentissage des élèves.

2. La pédagogie :

- **Définition:** La pédagogie se réfère à l'ensemble des méthodes, des pratiques et des principes utilisés par les enseignants pour guider et faciliter l'apprentissage des élèves. Elle englobe l'ensemble des actions éducatives visant à favoriser le développement global des apprenants.
- **Portée :** La pédagogie est un domaine plus large qui englobe divers aspects de l'éducation, y compris les méthodes d'enseignement, les théories de l'apprentissage, la gestion de classe, et la relation enseignant-élève.
- **Approche :** La pédagogie adopte une approche plus globale et centrée sur l'apprenant, en mettant l'accent sur les besoins, les intérêts et les capacités individuels des élèves

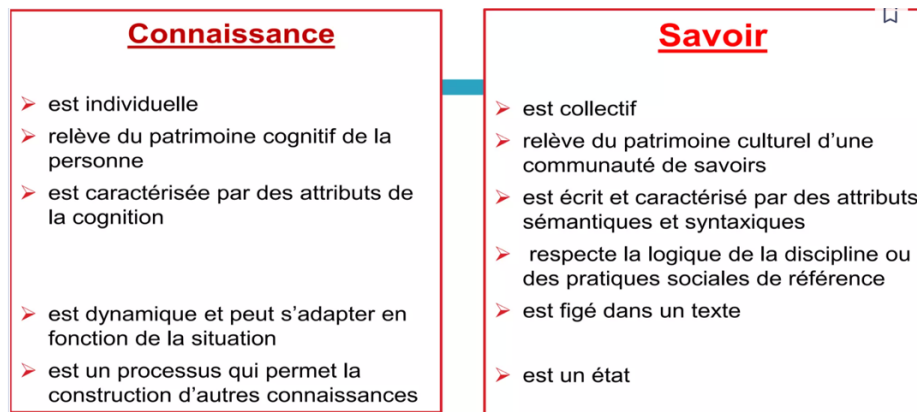
La pédagogie et la didactique devraient donc être considérées dans leur relation de complémentarité. Ceci est d'autant plus évident que l'enseignant, dans l'exercice de son métier, accomplit deux tâches : il gère, d'une part, le curriculum et de l'autre le groupe-classe.

II. Différence entre savoir et connaissance:

- **Le savoir** est une donnée, un concept, une procédure ou une méthode qui existe à un temps donné hors de tout sujet connaissant et qui est généralement codifié dans des ouvrages de référence. L'acquisition d'un savoir suppose un processus continu d'assimilation et d'organisation de connaissances par l'élève.
- **La connaissance** est propre et différente à chaque individu. En effet, c'est ce que l'élève a retenu, compris, construit du savoir correspondant. C'est donc ce que l'élève s'est approprié. Mais il arrive que les connaissances soient erronées.

Une connaissance vit dans une situation, alors que le savoir vit dans une institution. Pour définir une connaissance, il faut décrire les situations qui la caractérisent. Pour définir un savoir, il faut déterminer l'institution qui le produit et le légitime, ce qui conduit parfois à considérer plusieurs institutions et leurs éventuels conflits.

Si un savoir existe dans une institution, c'est qu'il a été rencontré comme une connaissance en situation puis reconnu comme utile, formulé, formalisé, validé, mémorisé et qu'il a acquis un statut institutionnel : c'est le processus d'institutionnalisation, envisagé comme une transformation qui légitime tout savoir dans une institution.

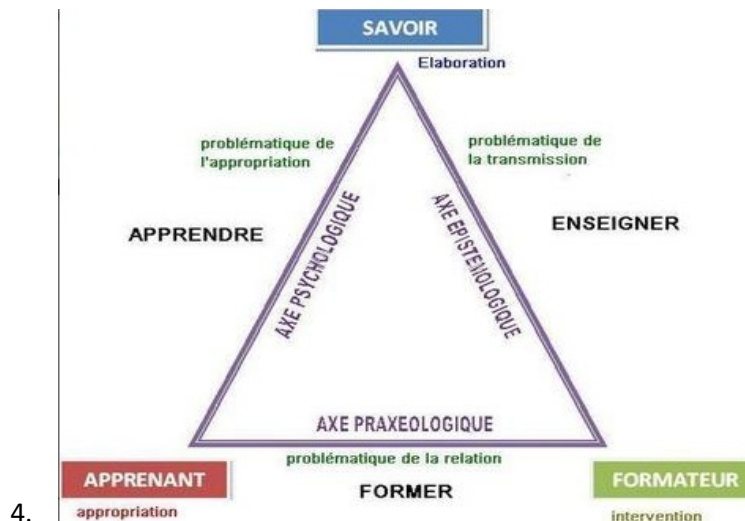


III. Triangle didactique

1. Définition

Le triangle didactique, développé par Yves Chevallard au début des années 80s, est un modèle pour représenter les interactions entre les trois éléments clés de l'acte d'enseigner : enseignant, élève et le savoir

1. **Enseignant** : L'enseignant est l'acteur principal du processus d'enseignement. Il est chargé de planifier, de présenter et de faciliter l'apprentissage des élèves. Les responsabilités de l'enseignant comprennent la conception de programmes d'études, la préparation de matériel pédagogique, la présentation de contenu, la gestion de classe, l'évaluation des élèves et la rétroaction.
2. **Élève** : L'élève est le destinataire de l'enseignement. Il est actif dans le processus d'apprentissage et est chargé d'assimiler, de comprendre et d'appliquer les connaissances et les compétences présentées par l'enseignant. Les élèves ont des besoins, des styles d'apprentissage et des niveaux de compétence différents, ce qui nécessite une différenciation de l'enseignement pour répondre à leurs besoins individuels.
3. **Savoir** : Le contenu de l'enseignement comprend les connaissances, les concepts, les compétences et les valeurs que l'enseignant souhaite transmettre aux élèves. Il peut être organisé en programmes d'études, en leçons, en unités ou en cours spécifiques. Le contenu de l'enseignement est sélectionné en fonction des objectifs pédagogiques, des normes éducatives et des besoins des élèves.



2. Les trois axes du triangle didactique : praxéologie, épistémologie et psychologie

Pour approfondir la compréhension de ces interactions, Chevallard propose d'analyser le triangle didactique à travers trois axes : praxéologique, épistémologique et psychologique.

1. Axe praxéologique :

Cet axe se focalise sur les **gestes professionnels** du professeur et les **savoirs pratiques** mobilisés dans l'enseignement. Il s'agit de la dimension **actionnelle** du triangle didactique.

Questions clés:

- Quelles sont les actions et les décisions que le professeur prend en classe ?
- Comment le professeur organise-t-il les situations d'apprentissage ?
- Quelles sont les ressources et les outils que le professeur utilise ?

Concepts importants:

- Situations d'apprentissage
- Gestion de classe

2. Axe épistémologique :

Cet axe se focalise sur les **savoirs disciplinaires** à enseigner et sur leur **structure interne**. Il s'agit de la dimension **savante** du triangle didactique.

Questions clés:

- Quels sont les concepts fondamentaux de la discipline ?
- Comment les savoirs disciplinaires sont-ils organisés ?
- Quelles sont les démarches scientifiques et les méthodes de recherche propres à la discipline ?

Concepts importants:

- La transposition didactique.
- La trame conceptuelle.

- Les niveaux de formulation.
- Les champs conceptuels.

3. Axe psychologique :

Cet axe se focalise sur les **apprentissages** des élèves et sur les **processus cognitifs** en jeu. Il s'agit de la dimension **cognitive** du triangle didactique.

Questions clés:

- Comment les élèves construisent-ils leurs connaissances ?
- Quels sont les facteurs qui influencent l'apprentissage ?
- Quelles sont les difficultés que les élèves rencontrent ?

Concepts importants:

- Représentations
- Les obstacles à l'apprentissage.
- Styles d'apprentissage

Le triangle didactique met en évidence les interactions dynamiques entre l'enseignant, l'élève et le contenu de l'enseignement. Chaque élément influence et est influencé par les autres, créant un environnement d'apprentissage complexe et dynamique. Par exemple, l'enseignant doit adapter ses méthodes d'enseignement en fonction des besoins et des capacités des élèves, tout en sélectionnant et en présentant le contenu de manière appropriée. De même, les élèves réagissent au contenu de l'enseignement et aux méthodes d'enseignement de l'enseignant, ce qui peut influencer leur engagement et leur apprentissage. En comprenant ces interactions, les enseignants peuvent concevoir des environnements d'apprentissage efficaces qui favorisent la réussite des élèves.

L'analyse du triangle didactique à travers les trois axes praxéologique, épistémologique et psychologique permet de mieux comprendre la complexité de l'acte d'enseigner et de concevoir des situations d'apprentissage plus efficaces.

IV. Transposition didactique :

La transposition didactique, introduite par Gaston Bachelard et développée par Yves Chevallard, fait référence au processus par lequel un contenu issu d'un domaine de connaissance (le savoir savant) est transformé et adapté pour être enseigné dans un contexte éducatif (le savoir enseigné). La transposition didactique est donc le travail de transformation des connaissances savantes en connaissances à enseigner dans les manuels, puis en connaissances à enseigner dans les salles de classe.

Les deux étapes de la transposition didactique

Selon Develay (1992), le processus de transposition didactique comprend deux étapes, la transposition interne et la transposition externe.

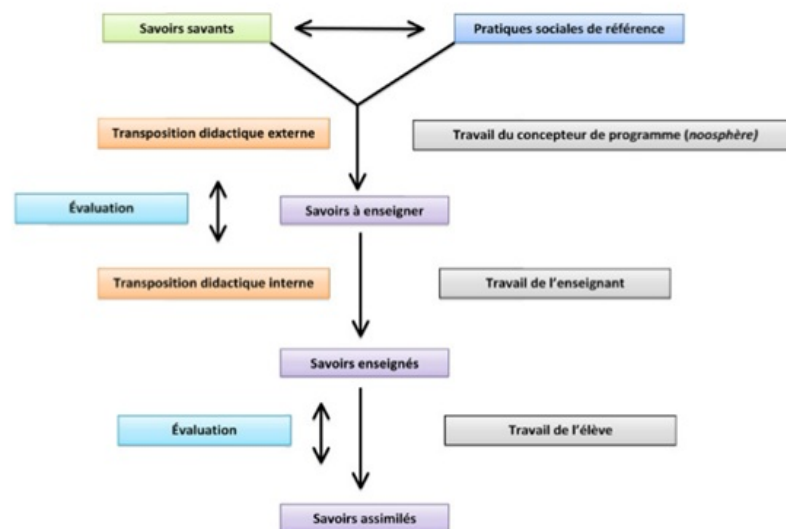
▪ La transposition didactique externe

La transposition externe est l'étape au cours de laquelle les connaissances savantes (ou connaissances expertes) sont transformées en connaissances à enseigner (les connaissances identifiées dans le programme scolaire ou manifestées dans les manuels). Elle est régie par ce

qu'appelle Chevallard (1985) « **noosphère** », littéralement « la sphère où l'on pense ». La noosphère est donc l'ensemble des personnes qui pensent les contenus d'enseignement : les universitaires, les auteurs de manuels, les inspecteurs scolaires, les représentants de la société, le président d'une association de parents d'élèves, et les représentants du monde politique

▪ La transposition didactique interne

La transposition interne est l'étape au cours de laquelle les connaissances qui doivent être enseignées sont transformées en connaissances qui peuvent être enseignées dans les classes par l'enseignant. Ce processus de transposition implique des choix, des simplifications, des réorganisations et des adaptations de la part de l'enseignant pour rendre le contenu accessible et compréhensible pour les élèves.



V. Trame conceptuelle

Une trame conceptuelle est une représentation visuelle ou textuelle qui organise les concepts clés d'un domaine de connaissance ou d'un sujet spécifique. Elle permet de mettre en évidence les relations entre les différents concepts et de fournir une structure logique pour l'apprentissage et la compréhension.

Voici un exemple de trame conceptuelle pour le domaine de la programmation informatique :

- **Introduction à la programmation :**
 - Définition de la programmation
 - Importance de la programmation dans le monde moderne
- **Langages de programmation :**
 - Types de langages (compilés, interprétés)
 - Exemples de langages populaires (Python, Java, C++, etc.)
- **Structures de contrôle :**
 - Séquences d'instructions
 - Structures conditionnelles (if, else)

- Boucles (for, while)
- **Types de données :**
 - Entiers, flottants, chaînes de caractères
 - Structures de données (listes, tableaux, dictionnaires)
- **Fonctions et procédures :**
 - Définition et utilisation de fonctions
 - Passage de paramètres
 - Modularité et réutilisation du code
- **Programmation orientée objet :**
 - Concepts de base (classes, objets, méthodes)
 - Encapsulation, héritage, polymorphisme
- **Gestion des erreurs :**
 - Gestion des exceptions
 - Techniques de débogage
- **Algorithmique :**
 - Introduction aux algorithmes
 - Complexité algorithmique
 - Techniques de conception d'algorithmes (tri, recherche, etc.)
- **Développement d'applications :**
 - Méthodologie de développement logiciel
 - Processus de conception, développement, test et déploiement

Cette trame conceptuelle offre une vue d'ensemble des principaux concepts de la programmation informatique, organisés de manière logique et hiérarchique. Elle peut servir de guide pour structurer l'enseignement et l'apprentissage de la programmation, en mettant en évidence les relations entre les différents concepts et en fournissant un cadre cohérent pour l'acquisition de connaissances et de compétences dans ce domaine.

VI. Niveau de formulation d'un concept :

Le niveau de formulation d'un concept fait référence au degré de complexité et d'abstraction avec lequel un concept est présenté aux élèves. Il s'agit de la manière dont le concept est exprimé et expliqué, ainsi que des exemples et des illustrations utilisés pour le clarifier.

Les concepts peuvent être formulés à différents niveaux de complexité en fonction des besoins des élèves et des objectifs d'apprentissage. Par exemple, un concept peut être présenté de manière simplifiée pour les élèves débutants, puis approfondi progressivement à mesure que leur compréhension se développe.

La formulation d'un concept peut également varier en fonction du domaine de connaissance, du contexte éducatif et des caractéristiques des élèves. Certains concepts peuvent nécessiter une formulation plus abstraite et théorique, tandis que d'autres peuvent être mieux compris avec des exemples concrets et des applications pratiques.

Prenons par exemple le concept de "boucle" en programmation informatique. Une boucle est une structure de contrôle qui permet d'exécuter un bloc d'instructions de manière répétée tant qu'une condition donnée reste vraie.

- **Niveau de formulation bas** : Dans un cours d'introduction à la programmation pour les débutants, un enseignant pourrait expliquer le concept de boucle de manière simplifiée en utilisant un langage accessible. Par exemple : "Une boucle en programmation est une instruction qui permet de répéter plusieurs fois une série d'actions. C'est comme lorsque vous répétez une tâche plusieurs fois dans un jeu vidéo."
- **Niveau de formulation intermédiaire** : Dans un cours de programmation pour des apprenants plus avancés, l'enseignant pourrait fournir une explication plus détaillée et précise : "En programmation, une boucle est une structure de contrôle qui permet de répéter un bloc d'instructions plusieurs fois tant qu'une condition spécifiée reste vraie. Il existe différents types de boucles, notamment la boucle "for", la boucle "while" et la boucle "do-while", chacune ayant ses propres caractéristiques et utilisations."
- **Niveau de formulation élevé** : Dans un cours universitaire avancé sur l'algorithmique, l'enseignant pourrait aborder le concept de boucle d'une manière plus théorique et analytique : "En informatique, une boucle est un mécanisme fondamental de contrôle de flux qui permet de répéter un bloc d'instructions jusqu'à ce qu'une condition de terminaison soit atteinte. La compréhension de la complexité temporelle des algorithmes itératifs et des techniques d'optimisation de boucle est essentielle pour concevoir des programmes efficaces et performants."

Dans chaque exemple, le concept de boucle est présenté à un niveau de formulation différent en fonction du niveau de compétence et de familiarité des apprenants avec le sujet. L'enseignant adapte ainsi la formulation du concept pour répondre aux besoins spécifiques de son public cible.

VII. Les erreurs

L'erreur est généralement considérée soit comme un écart entre la performance réalisée (la réponse) et un but attendu (ou une norme définie), soit comme le processus responsable de cet écart.

Depuis quelques années, les recherches en éducation, et plus particulièrement en didactique, ont permis de passer d'une conception négative des erreurs donnant lieu à sanction, à une conception nouvelle où celles-ci apparaissent plutôt comme un indice de la manière dont fonctionne le processus d'apprentissage et comme un témoin précieux pour repérer les difficultés des élèves. En concevant différemment l'erreur, il est possible de renouveler la compréhension de ce qui se passe dans la classe pour rendre plus efficace l'enseignement.

Il existe plusieurs catégories de sources d'erreurs. On cite :

- **Erreurs liées à l'élève**

Limitation du sujet à un moment de son développement psychogénétique.

- **Erreurs liées au rapport entre l'élève et le savoir**

Elles ont pour origine une conception de l'élève, un modèle plus ou moins implicite dans sa tête, et sont dues à un savoir ancien qui fait obstacle ; il s'agit donc d'une connaissance antérieure, mal appliquée ou appliquée en dehors de son domaine de validité.

- **Erreurs liées aux choix didactiques de l'enseignant**

Certains choix faits dans l'enseignement renforcent les obstacles culturels

- **Erreurs liées à la relation didactique**

Ce sont des erreurs provenant de ce que l'élève imagine des attentes du professeur : un problème a toujours une solution, il faut utiliser toutes les données du texte, si on vient d'étudier tel théorème il faut l'utiliser dans la solution. L'élève n'a pas compris ce qu'on lui demande (pas de sens relativement au savoir ou à ses connaissances) mais donne quand même une réponse car il faut toujours répondre (c'est mieux que de rendre copie blanche, ou l'élève pense gagner quelques points...). On parle d'erreurs liées au contrat didactique.

Exemples :

- **Généralisation excessive** : Un élève peut généraliser incorrectement un concept informatique à toutes les situations sans tenir compte des exceptions ou des conditions spécifiques. Par exemple, ils peuvent croire que toutes les boucles de programmation fonctionnent de la même manière, sans reconnaître les nuances entre différents types de boucles.
- **Application incorrecte des règles** : Un élève peut appliquer une règle ou une méthode de résolution qu'ils ont apprise dans un contexte différent, ce qui peut conduire à des résultats incorrects. Par exemple, ils peuvent utiliser une syntaxe incorrecte dans un langage de programmation parce qu'ils l'ont apprise dans un autre langage.
- **Interprétation erronée des concepts** : Les élèves peuvent interpréter incorrectement un concept informatique en raison d'une mauvaise compréhension des définitions ou des principes sous-jacents. Par exemple, ils peuvent confondre les concepts de l'algorithme et du programme, pensant qu'ils sont interchangeables.
- **Utilisation inappropriée des outils** : Les élèves peuvent utiliser incorrectement les outils informatiques en raison d'une mauvaise compréhension de leurs fonctionnalités ou de leurs limitations. Par exemple, ils peuvent utiliser une fonctionnalité avancée dans un logiciel sans comprendre comment elle fonctionne, ce qui peut entraîner des résultats imprévus ou des erreurs.
- **Inférences incorrectes** : Les élèves peuvent tirer des conclusions incorrectes ou inexactes à partir des données ou des résultats qu'ils obtiennent lors de l'analyse de problèmes informatiques. Par exemple, ils peuvent mal interpréter les résultats d'une expérience de programmation et en tirer des conclusions erronées sur le fonctionnement du programme.

VIII. Les représentations/conceptions initiales

1. Définition

Les représentations initiales, appelée aussi conceptions initiales, sont des systèmes explicatifs possédés par chacun et qui nous permettent de comprendre le monde qui nous entoure. Ces conceptions se construisent dès la naissance et ne sont pas souvent en adéquation avec la connaissance. Ce sont souvent de fausses idées, un modèle mental que l'on se fait sur un sujet avant que tout apprentissage n'ait été fait. Ces modèles présents avant tout apprentissage expliquent en partie qu'ils soient souvent erronés et ils peuvent persister après apprentissage.

Les représentations sont personnelles mais pas uniques. En effet, plusieurs apprenants peuvent avoir une même représentation. En effet, les représentations sont influencées par différents registres tels que le savoir

scolaire étudié antérieurement, des sources textuelles ou documentaires mais encore l'environnement, les croyances religieuses ou encore des sources audiovisuelles.

Exemples :

Les fichiers comme des objets physiques : Certains élèves peuvent avoir du mal à comprendre que les fichiers sur un ordinateur ne sont pas des objets physiques tangibles, mais plutôt des ensembles de données stockées sous forme numérique sur un support de stockage.

Les virus informatiques comme des êtres vivants : Certains élèves peuvent avoir une conception erronée selon laquelle les virus informatiques sont des entités vivantes capables de se déplacer et de se reproduire, plutôt que comme des programmes malveillants conçus pour endommager les systèmes informatiques.

Influence des stéréotypes de genre: Une élève pense que l'informatique est un domaine réservé aux garçons et ne se voit pas exercer un métier dans ce secteur.

2. Émergence et évolution des représentations :

Les représentations émergent dès le plus jeune âge à travers les interactions de l'élève avec son environnement. Ils se construisent à partir des expériences vécues, des interactions sociales, des médias et des influences culturelles. Par exemple, un élève peut se représenter l'informatique comme un domaine réservé aux experts en raison des messages médiatiques ou des stéréotypes sociaux :

- **Influence des expériences éducatives :** Les représentations des élèves sont influencées par leurs expériences éducatives. Les méthodes d'enseignement, les interactions avec les enseignants et les pairs, ainsi que les contenus enseignés contribuent à façonner et à modifier leurs représentations. Par exemple, des expériences positives en classe d'informatique peuvent aider un élève à percevoir ce domaine comme accessible et stimulant.
- **Révision et ajustement :** Les représentations des élèves ne sont pas statiques ; elles peuvent évoluer et se modifier au fil du temps. Lorsque les élèves sont confrontés à de nouvelles informations, à des expériences contradictoires ou à des défis intellectuels, ils peuvent réviser leurs représentations pour mieux correspondre à leur compréhension actuelle du sujet. Par exemple, un élève peut réviser sa perception de l'informatique après avoir réussi à résoudre un problème de programmation complexe.
- **Construction sociale des représentations :** Les représentations des élèves sont également influencées par les interactions sociales avec leurs pairs, leurs enseignants et d'autres membres de la communauté éducative. Les discussions en classe, les projets de groupe et les activités collaboratives permettent aux élèves de partager leurs perspectives, de confronter leurs idées et de construire collectivement leur compréhension du sujet.
- **Continuité et changement :** Bien que les représentations des élèves puissent évoluer au fil du temps, certaines tendances ou schémas de pensée peuvent persister. Par exemple, un élève peut continuer à voir l'informatique comme un domaine complexe et intimidant malgré des expériences éducatives positives, en raison de représentations préexistantes ou de facteurs externes tels que des stéréotypes sociaux.

IX. Obstacles

Prendre en considération les obstacles dans une approche d'enseignement implique une compréhension, même partielle, des diverses sources de ces obstacles. En effet, cette compréhension permet d'identifier ces obstacles de manière plus précise et de les aborder de manière plus efficace sur le plan pédagogique. Cependant, une connaissance des origines de ces obstacles requiert des compétences spécifiques de la part de l'enseignant, lesquelles peuvent être acquises par le biais de formations dispensées dans les centres de formation des enseignants. Cela est réalisable si les conditions sont réunies pour que la formation puisse réellement renforcer ces compétences chez l'enseignant.

Il est irréaliste de prétendre connaître toutes les origines potentielles des obstacles à l'apprentissage, mais il est possible de dresser une liste non exhaustive des origines possibles et probables de ces obstacles. Dans cette optique, on peut mentionner les sources suivantes :

- Obstacles liés au langage ;
- Obstacles liés à la simplification du savoir à des fins d'enseignement ;
- Obstacles liés à la nature intrinsèque du savoir lui-même ;
- Obstacles liés à des notions préalables nécessaires à la compréhension du concept étudié ;
- Obstacles générés par les manuels et les livres scolaires ;
- Obstacles découlant des modèles proposés par l'enseignant ou présentés dans les ouvrages de référence ;
- Obstacles liés au style d'apprentissage de l'apprenant ;
- Obstacles associés aux stratégies pédagogiques mises en œuvre par l'enseignant.

Astolfi (1996) nous propose six étapes nécessaires pour la prise en compte des présentations :

- Les entendre par une écoute positive de ce qu'expriment les élèves.
- Les comprendre en postulant que les erreurs ne sont pas fortuites mais méritent d'être analysées.
- Les faire identifier : Vu le fonctionnement inconscient des représentations, la prise de conscience par chacun contribuant déjà à leur évolution.
- Les faire comparer ce qui favorise la décentration des points de vue, et révèle aux élèves une diversité qu'ils n'imaginent pas dans les idées en présence dans la classe pour expliquer un même phénomène.
- Les faire discuter en provoquant des conflits sociocognitifs dont la psychologie indique que ce sont d'importants leviers du développement intellectuel.
- Les suivre en surveillant leur évolution à court et moyen terme, au long de la scolarité obligatoire et d'abord au cours d'une même année

Il faut noter ici que ces processus de prise en compte des représentations évoque certaines objections dues à la gestion du temps didactique face à des programmes chargés ce qui demande un temps supplémentaire que l'institution scolaire ne peut pas accorder.

X. Objectif-obstacle :

L'objectif-obstacle, concept introduit par JEAN-LOUIS MARTINAND en 1982, fait référence à une situation dans laquelle un individu est confronté à un problème ou à un défi qui nécessite une résolution cognitive. Cet obstacle peut prendre différentes formes, telles qu'un problème à résoudre, une tâche complexe à accomplir ou une contradiction entre les connaissances préexistantes et de nouvelles informations.

L'objectif-obstacle est un moteur puissant de l'apprentissage car il incite les élèves à mobiliser leurs connaissances, leurs compétences et leurs stratégies de résolution de problèmes pour surmonter le défi.

Astolfi (1989) a présenté un processus permettant de concrétiser le concept d'objectif-obstacle. Ce processus se décompose en les étapes suivantes :

- Repérer les obstacles à l'apprentissage (dont les représentations font partie), sans les minorer ni les sur valoriser.
- Définir inversement, et de manière plus dynamique, le progrès intellectuel correspondant à leur éventuel franchissement.
- Sélectionner, parmi la diversité des obstacles repérés, celui (ou ceux) qui paraît franchissable au cours d'une séquence, produisant un progrès intellectuel décisif.
- Se fixer comme objectif le dépassement de cet obstacle jugé franchissable.
- Traduire cet objectif en termes opérationnels selon les méthodologies classiques de formulation des objectifs.
- Construire un dispositif (ou plusieurs), cohérent avec l'objectif, ainsi que des procédures de remédiation en cas de difficulté.

Exemple :

- **Objectif :** Les élèves apprendront le concept d'instructions conditionnelles (si-sinon) en créant un jeu vidéo simple, tel qu'un jeu de devinettes.
- **Obstacle :** L'enseignant divise l'objectif principal en étapes plus petites et plus accessibles. Par exemple, les étapes pourraient inclure la création d'une fenêtre de jeu, la définition des règles du jeu et la programmation des interactions dynamiques avec l'utilisateur que les élèves peuvent avoir du mal à mettre en œuvre sans connaître les structures conditionnelles.
- **Franchissement de l'obstacle :** les élèves découvrent le concept d'instructions conditionnelles (si - sinon) comme une méthode pour ajouter des choix et des interactions dynamiques dans le jeu. L'enseignant explique comment ces structures de contrôle peuvent être utilisées pour créer différentes réponses en fonction des actions de l'utilisateur.

XI. Conflit sociocognitif :

Le conflit sociocognitif dans l'enseignement fait référence à une situation dans laquelle les élèves sont confrontés à des opinions, des perspectives ou des idées divergentes au sein d'un groupe d'apprentissage. En effet, face à un problème ou une question, chaque membre d'un groupe d'élèves a au démarrage une représentation qui lui est propre du problème ou de la question. Étant dans une situation d'échange et de communication entre eux, chacun peut émettre des idées, proposer sa vision du problème. Ce partage de point

de vue amène chacun à faire évoluer sa représentation de départ et à aboutir à une représentation nouvelle, si ce n'est commune au groupe, en tous les cas différentes des positions de chaque interlocuteur au départ.

Ce conflit peut émerger lors de discussions en classe, de débats, de collaborations sur des projets ou de confrontations d'idées entre pairs. Voici quelques exemples de la manière dont le conflit sociocognitif peut se manifester dans l'enseignement :

- **Débats en classe** : L'enseignant organise un débat sur un sujet controversé, tel que l'impact des réseaux sociaux sur la société. Les élèves expriment des opinions divergentes et argumentent leurs points de vue, ce qui crée un conflit sociocognitif stimulant la réflexion critique et la remise en question des croyances préexistantes.
- **Collaboration sur des projets** : Les élèves travaillent en groupe sur un projet de recherche. Ils doivent fusionner différentes perspectives et idées pour aboutir à un résultat final, ce qui peut entraîner des désaccords et des discussions sur la meilleure approche à adopter.
- **Analyse de sources contradictoires** : Les élèves sont invités à examiner des sources d'information contradictoires sur un sujet donné, telles que des articles de presse ou des études de recherche. Ils doivent évaluer la crédibilité des sources et résoudre les contradictions, ce qui favorise la confrontation d'idées et la résolution de problèmes.
- **Critique de pairs** : Les élèves présentent leurs travaux à leurs pairs pour recevoir des commentaires et des critiques constructives. Les divergences d'opinions sur la qualité du travail ou sur les approches utilisées peuvent susciter un conflit sociocognitif qui encourage les élèves à réévaluer et à améliorer leur travail.

En favorisant le conflit sociocognitif dans l'enseignement, les enseignants peuvent encourager les élèves à explorer de nouvelles idées, à remettre en question leurs croyances préexistantes et à développer leur pensée critique. Cependant, il est important que ce conflit soit géré de manière constructive, en créant un environnement où les élèves se sentent en sécurité pour exprimer leurs opinions et où les désaccords sont traités avec respect et ouverture d'esprit.

XII. Modèles didactiques

La notion de modèle didactique renvoie à une modélisation des processus d'apprentissage. Michel Meirieu, pédagogue français, distingue quatre grands types d'opérations dans les modèles didactiques :

1. **Induction** : L'induction est un processus par lequel les élèves découvrent ou construisent de nouvelles connaissances à partir d'exemples concrets ou d'observations. Dans cette approche, l'enseignant propose des situations-problèmes, des expériences ou des cas concrets aux élèves, qui sont ensuite invités à généraliser, à formuler des hypothèses ou à tirer des conclusions à partir de ces exemples. L'induction favorise l'engagement actif des élèves dans leur apprentissage et encourage le développement de leur pensée critique et de leur capacité à résoudre des problèmes.
 - Un exemple d'induction en informatique pourrait être une leçon sur les boucles en programmation. Plutôt que d'expliquer d'abord les concepts de boucles de manière théorique, l'enseignant pourrait présenter aux élèves quelques exemples de problèmes réels où l'utilisation de boucles serait utile. Les élèves pourraient ensuite explorer ces exemples, découvrir les schémas communs et les solutions récurrentes, et induire par eux-mêmes les principes fondamentaux des boucles en programmation.

2. **Déduction** : La déduction est un processus par lequel les élèves appliquent des règles, des principes ou des modèles généraux pour résoudre des problèmes spécifiques. Dans cette approche, l'enseignant présente d'abord les concepts ou les principes généraux, puis guide les élèves dans l'application de ces concepts à des situations particulières. La déduction favorise la compréhension des principes fondamentaux et la résolution systématique de problèmes.
 - Un exemple de déduction en informatique pourrait être une leçon sur la syntaxe d'un langage de programmation. L'enseignant pourrait commencer par présenter les règles et les structures syntaxiques de base du langage, puis guider les élèves dans l'application de ces règles pour écrire des programmes simples. Les élèves utiliseraient ensuite ces principes pour déduire la structure syntaxique correcte à utiliser dans des situations similaires.
3. **Dialectique** : La dialectique est un processus par lequel les élèves explorent les contradictions, les oppositions ou les tensions entre différentes idées, perspectives ou théories. Dans cette approche, l'enseignant encourage les élèves à analyser les points de vue divergents, à remettre en question les présupposés et à développer leur pensée critique en confrontant différentes opinions. La dialectique favorise le développement de la capacité des élèves à penser de manière complexe et à considérer les multiples aspects d'une question ou d'un problème.
 - Un exemple de dialectique en informatique pourrait être un débat sur l'éthique de l'intelligence artificielle. Les élèves pourraient être divisés en groupes représentant différentes perspectives, telles que les avantages économiques, les implications sociales, les préoccupations éthiques, etc. Ils discuteraient ensuite des arguments pour et contre chaque perspective, en confrontant leurs opinions et en cherchant à comprendre les points de vue divergents.
4. **Divergence** : La divergence est un processus par lequel les élèves génèrent de nouvelles idées, perspectives ou solutions créatives à partir d'une variété de stimuli ou de points de départ. Dans cette approche, l'enseignant encourage la pensée divergente en proposant des tâches ouvertes, des exercices de brainstorming ou des activités de résolution de problèmes qui permettent aux élèves d'explorer différentes possibilités, de faire des associations libres et d'exprimer leur créativité. La divergence favorise l'innovation, l'originalité et la flexibilité de la pensée.
 - Un exemple de divergence en informatique pourrait être une activité de résolution créative de problèmes. Par exemple, les élèves pourraient être invités à concevoir une nouvelle application mobile pour résoudre un problème spécifique dans leur communauté. Ils exploreraient différentes idées, généreraient des solutions innovantes, et exprimeraient leur créativité en proposant des fonctionnalités uniques et des designs originaux pour leur application.

Théorie de la situation didactique

I. Introduction

La Théorie de la Situation Didactique (TSD) est une théorie développée par Guy Brousseau dans le domaine de l'éducation mathématique. Elle vise à comprendre et à améliorer le processus d'enseignement et d'apprentissage des mathématiques en mettant l'accent sur la conception de situations didactiques appropriées. Voici un aperçu des principes fondamentaux de la TSD :

- **Situation Didactique** : La notion centrale de la TSD est la "situation didactique", qui représente un cadre structuré dans lequel se déroule l'interaction entre l'enseignant et les élèves pour construire des connaissances mathématiques. Cette situation est conçue pour favoriser un apprentissage actif et significatif en fournissant des tâches et des activités stimulantes, pertinentes et contextualisées. La situation didactique est caractérisée par sa complexité et sa richesse, car elle doit être suffisamment ouverte pour permettre l'émergence de multiples stratégies d'apprentissage tout en restant suffisamment cadrée pour atteindre les objectifs pédagogiques spécifiques.
- **Contractualisation** : La contractualisation dans une situation didactique implique un accord tacite entre l'enseignant et les élèves sur les objectifs, les règles et les attentes de la tâche à accomplir. Ce contrat définit les droits et les devoirs de chacun, ainsi que les règles de fonctionnement de la classe. Il est essentiel pour établir un climat de confiance et de coopération propice à l'apprentissage.
- **Instrumentation** : L'instrumentation se réfère aux outils, aux supports et aux ressources utilisés par l'enseignant pour faciliter l'apprentissage des élèves au sein de la situation didactique. Cela peut inclure des manipulables, des modèles, des logiciels informatiques, des exercices, des exemples, des questionnaires, etc. L'instrumentation est soigneusement choisie pour répondre aux besoins spécifiques des élèves et pour favoriser leur engagement et leur compréhension.
- **Validation** : La validation consiste à évaluer la pertinence et l'efficacité des situations didactiques pour atteindre les objectifs d'apprentissage spécifiques. Cela implique de recueillir des données sur les interactions entre l'enseignant et les élèves, sur les performances des élèves et sur les processus d'apprentissage en cours. Ces informations sont utilisées pour ajuster et améliorer les situations didactiques afin d'optimiser les résultats d'apprentissage.
- **Transposition didactique** : La transposition didactique est le processus par lequel les connaissances mathématiques sont sélectionnées, organisées et adaptées pour être enseignées dans le contexte de la situation didactique. Cela implique souvent de traduire les concepts abstraits en représentations concrètes et accessibles aux élèves, ainsi que de structurer les contenus pour les rendre progressivement plus complexes et abordables.
- **Référentiel** : Le référentiel désigne l'ensemble des connaissances mathématiques et des compétences visées par la situation didactique. Il fournit un cadre conceptuel pour la conception et l'évaluation

des activités d'apprentissage, ainsi qu'un point de référence pour évaluer les progrès des élèves et identifier les lacunes dans leur compréhension.

En combinant ces différents éléments, la Théorie de la Situation Didactique offre un cadre complet pour la conception, la mise en œuvre et l'évaluation des situations d'apprentissage en mathématiques. Elle met l'accent sur l'importance de créer des environnements d'apprentissage riches et stimulants, où les élèves sont activement engagés dans la construction de leur propre savoir mathématique.

II. Contrat didactique :

Le contrat didactique est un concept central dans la théorie des situations didactiques (TSD). Il désigne un ensemble de règles implicites ou explicites qui définissent les rôles et les responsabilités de l'enseignant et des élèves dans le processus d'enseignement et d'apprentissage. Par exemple, il peut préciser que l'enseignant est responsable de la transmission des connaissances et de la gestion de la classe, tandis que les élèves sont responsables de leur propre apprentissage et de leur participation active en classe. Le contrat didactique implique également des attentes mutuelles entre l'enseignant et les élèves. Cela peut inclure des attentes concernant l'engagement des élèves dans les activités d'apprentissage, la qualité du travail fourni, la coopération en classe, etc.

Éléments clés du contrat didactique:

- **Les tâches:** Ce que l'élève est censé faire et apprendre.
- **Les techniques:** Les moyens que l'élève est censé utiliser pour accomplir les tâches.
- **Les résultats:** Ce que l'enseignant attend de l'élève.
- **Les sanctions:** Les conséquences positives ou négatives du respect ou du non-respect du contrat.

Types de contrats didactiques:

- **Explicite:** Les règles du contrat sont clairement définies et communiquées aux élèves.
- **Implicite:** Les règles du contrat ne sont pas clairement définies et les élèves les découvrent au fil du temps.
- **Négocié:** Les règles du contrat sont négociées entre l'enseignant et les élèves.

Le contrat didactique peut être ajusté en fonction des obstacles rencontrés. L'enseignant peut proposer des aides et des étayages temporaires pour permettre aux élèves de progresser malgré les obstacles.

III. CONTRAT DIDACTIQUE : TOPOGENÈSE ET CHRONOGENÈSE

Chevallard (1991) a redécrit la notion en montrant comment le contrat didactique peut s'exprimer de la manière suivante :

- dans le processus didactique il y a une élémentation du savoir, sous laquelle un corps de connaissances se recomposait, par l'enseignement, en une séquence graduée selon un ordre logique, du simple au complexe. Enseigner, c'est donc parcourir avec les élèves une séquence, une suite orientée d'objets de savoir. Cette disposition du savoir sur l'axe du temps, c'est le temps didactique, aussi appelé *chronogenèse*.
- à chaque instant de la chronogenèse, le professeur et les élèves occupent un lieu précis, *un topos*, c'est-à-dire accomplissent un ensemble de tâches, dont certaines sont spécifiquement liées à la position de professeur, et d'autres à la position d'élève. Par exemple, dans le contrat didactique classique en

mathématiques, la démonstration appartient au topos du professeur, la recherche d'exercices appartient au topos de l'élève, et *une topogenèse* particulière est ainsi décrite. À chaque instant de la chronogenèse correspond un état de la topogenèse.

- **Milieu:** Ensemble des ressources et des contraintes à la disposition des élèves dans une situation d'apprentissage. la *mésogenèse* se définit comme l'évolution des objets meublant le milieu, et de leurs relations, par le biais d'interactions entre les apprenants et avec le professeur

Donc, on peut considérer le travail du professeur comme un travail de gestion:

- De la chronogenèse : il peut décider par exemple d'arrêter le travail dédié à une notion, pour passer à la notion « suivante » sur l'axe du temps)
- et de la topogenèse : il peut décider par exemple d'occuper moins d'espace symbolique dans la classe, à un instant précis, en mettant les élèves « à la recherche ».

Interactions entre ces éléments:

- Le contrat didactique définit les règles du jeu.
- La topogenèse et la chronogenèse organisent le déroulement de la situation d'apprentissage.
- Le milieu fournit les ressources et les contraintes qui influencent les interactions et les apprentissages.

IV. Situation didactique :

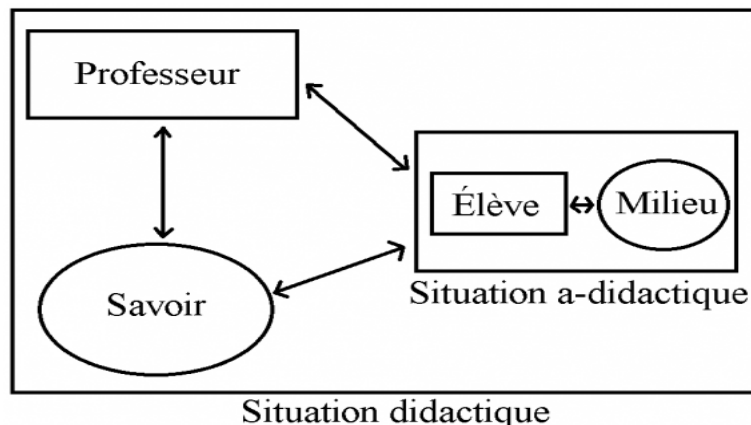
La théorie des situations didactiques est une approche de l'enseignement des mathématiques développée par le mathématicien et éducateur français Guy Brousseau dans les années 1970 et 1980. Elle est basée sur l'idée que l'apprentissage se produit lorsque les élèves sont placés dans des situations problématiques spécifiques et sont amenés à résoudre ces problèmes par eux-mêmes.

Selon Brousseau, une situation didactique comporte plusieurs phases :

- **la situation d' enrôlement :** un professeur doit concevoir un problème préservant le sens de la connaissance visée. Il s'agit de la situation fondamentale.
- **la situation de dévolution :** « La dévolution est l'acte par lequel l'enseignant fait accepter à l'élève la responsabilité d'une situation d'apprentissage (**adidactique**) ou d'un problème et accepte lui-même les conséquences de ce transfert »
- **la situation d'action :** (ou travail de recherche et manipulation, d'élucidation d'une situation problématique à résoudre)
- **la situation de formulation :** (des résultats des recherches et manipulations précédentes, de l'élucidation d'une situation problématique)
- **la situation de validation :** La phase de proposition de résolution du problème par l'élève correspond à une situation de validation s'il y a une phase de discussion avec un autre élève ou une équipe adverse afin de prouver ou réfuter sa conjecture.

Dans les phases d'action, de formulation et de validation, le professeur n'intervient pas pour proposer des connaissances, mais il s'assure que l'élève est en situation d'apprentissage. Ces règles de fonctionnement relatives à l'apprentissage du savoir constituent le contrat didactique dans une situation qualifiée *d'adidactique*.

- **la situation d'institutionnalisation :** Le professeur s'assure enfin, dans une partie d'institutionnalisation, de la décontextualisation et de la dépersonnalisation des connaissances acquises afin qu'elles puissent prendre le statut de savoir scolaire. C'est le processus dans et par lequel le professeur signifie aux élèves les savoirs ou les pratiques qu'il leur faut retenir comme les enjeux de l'apprentissage attendu



V. Situation-problème

En classe, l'enseignant « élabore une situation en fonction d'un objectif d'apprentissage, mais en dissimulant suffisamment cet objectif pour que l'élève ne puisse l'atteindre que par une adaptation personnelle à la situation ».

Il s'agit de situations didactiques construites autour d'un "problème" (une problématique à résoudre pour devenir compétent), le terme désignant un questionnement, une énigme, issue d'un objet, d'une observation, etc. (en général avec un support concret), dont la résolution nécessite l'investissement des élèves.

La situation-problème satisfait à plusieurs conditions : elle est nouvelle pour les élèves, possède un contexte pour l'inscrire dans la réalité, et peut même être authentique (on peut se baser sur un sujet d'actualité). Elle doit surtout être ouverte, à savoir qu'au départ, aucune solution n'est explicitée, c'est aux apprenants, ou à l'équipe d'enseignants qui les guide, de construire une démarche, les différentes étapes. Par ouvert, cela signifie qu'a priori le problème n'a pas une solution unique. De plus, ce problème doit représenter un challenge atteignable par les apprenants, à savoir qu'il doit nécessiter un apprentissage, un saut conceptuel, mais que les apprenants puissent résoudre ce problème avec un niveau d'effort acceptable.

Les élèves n'ont pas, au départ, tous les moyens de répondre à la question. Ils doivent tout d'abord s'appropriier le questionnement (dévolution) et mettre en œuvre leurs connaissances et leur ingéniosité pour trouver 'une' solution (en passant par une expérience concrète si besoin).

La situation didactique peut être choisie par l'enseignant de façon à ce que le problème révèle un conflit (cognitif) et que la résolution corresponde donc au franchissement d'un obstacle. Enfin, l'activité n'est pas nécessairement individuelle, mais peut reposer sur un travail de groupe pouvant faire apparaître des conflits (socio-cognitifs).

VI. Variables didactiques :

On parle de variable didactique en situation d'apprentissage lorsque l'enseignant introduit des variantes déclenchant une modification dans les stratégies des élèves et amènent à adopter de nouvelles règles. Ces adaptations peuvent être décidées en cours d'apprentissage, elles permettent alors soit de simplifier, soit de complexifier une action, une tâche, favorisant la construction des savoirs.

Quand un enseignant propose une situation qu'il a construite ou choisie dans un ouvrage, un certain nombre de paramètres sont à sa disposition pour lui permettre d'anticiper le travail des élèves. Ce sont les variables de la situation. Elles peuvent prendre, à son initiative différentes valeurs. Il existe plusieurs types de variables, de natures différentes :

- celles qui permettent d'adapter la tâche au niveau de la classe, aux capacités des élèves : ce sont les variables pédagogiques. C'est ainsi qu'est notamment régulée la différenciation ;
- La variation des valeurs de certaines variables entraîne une modification de la hiérarchie des stratégies de solutions, ce sont les variables didactiques. L'élève est conduit à changer sa procédure, cette dernière étant devenue trop longue, trop coûteuse ou erronée. Dans une situation donnée une variable didactique est un élément dont la variation est susceptible de modifier le processus de résolution que les élèves vont adopter.
- **Variables cognitives** : La nature des opérations arithmétiques à effectuer, La taille des nombres, Le contexte, le type de situation proposée
- **Variables ergonomiques** : La complexité de l'organisation des calculs à effectuer, c'est-à-dire le nombre de données à traiter, prévoir et ordonner, Le nombre d'opérations de même nature à effectuer
- **Variables informationnelles** : La complexité formelle de l'énoncé (lisibilité des phrases, vocabulaire utilisé, l'ordre des informations), Les informations dites « inutiles » (nombre, forme...)

VII. Ressources didactiques pour l'enseignement et l'apprentissage

Les ressources didactiques sont des outils, supports et matériaux utilisés par les enseignants pour faciliter l'enseignement et l'apprentissage. Elles sont conçues pour aider les élèves à acquérir des connaissances, à développer des compétences et à atteindre les objectifs d'apprentissage fixés. Voici quelques exemples de ressources didactiques utilisées dans l'enseignement et l'apprentissage :

- **Manuels scolaires** : Les manuels scolaires sont des ressources didactiques essentielles qui fournissent aux élèves des informations structurées et organisées sur un sujet donné. Ils peuvent inclure des explications théoriques, des exercices pratiques, des activités et des exemples pour aider les élèves à comprendre et à appliquer les concepts enseignés.
- **Matériel pédagogique** : Le matériel pédagogique comprend une variété d'outils et de supports visuels utilisés pour illustrer et expliquer les concepts enseignés. Cela peut inclure des tableaux blancs, des tableaux interactifs, des affiches, des graphiques, des cartes, des modèles anatomiques, des maquettes, etc.
- **Logiciels éducatifs** : Les logiciels éducatifs sont des programmes informatiques conçus spécifiquement pour l'enseignement et l'apprentissage. Ils peuvent prendre la forme de jeux éducatifs, de simulations, de tutoriels interactifs, de quiz en ligne, etc., et sont souvent utilisés pour renforcer les compétences académiques et développer la maîtrise des outils technologiques.
- **Multimédia** : Les ressources multimédias, telles que les vidéos, les animations, les podcasts et les présentations interactives, sont utilisées pour présenter des informations de manière visuelle, auditive et dynamique. Elles captent l'attention des élèves, renforcent la compréhension des concepts et offrent des opportunités d'apprentissage multisensorielles.
- **Manipulables** : Les manipulables sont des objets physiques et tangibles utilisés pour manipuler et expérimenter des concepts abstraits. Ils sont particulièrement utiles dans les domaines des

mathématiques, des sciences et de la géométrie, où ils permettent aux élèves de visualiser et de comprendre les principes fondamentaux de manière concrète.

- **Ressources en ligne :** Les ressources en ligne, telles que les sites web éducatifs, les plateformes d'apprentissage en ligne, les cours virtuels, les forums de discussion et les blogs éducatifs, offrent un accès à une vaste gamme de contenus éducatifs et interactifs pour soutenir l'enseignement et l'apprentissage à distance.